On the (In)security of Approximate Computation Protocols from CKKS

Intak Hwang[1], Yisol Hwang[1], Miran Kim[2], Dongwon Lee[1], and Yongsoo Song[1]

[1] Seoul National University
{intak.hwang, yisol.hwang, dongwonlee95, y.song}@snu.ac.kr
[2] Hanyang University, Republic of Korea
miran@hanyang.ac.kr

Abstract. Secure Multi-Party Computation (MPC) enables collaborative and privacy-preserving computation over private inputs. Recent advances in Homomorphic Encryption (HE), particularly the CKKS scheme, have significantly improved the practicality of secure computation, making it suitable for real-world applications involving approximate arithmetic. However, the inherent errors introduced by CKKS pose substantial challenges in the design and analysis of CKKS-based secure computation protocols.

In this work, we investigate the correctness and security of existing CKKS-based MPC protocols. Most prior constructions rely on the noise smudging technique, in which each party independently adds exponentially large noise to the output to guarantee the security of the protocol. However, we show that this approach does not achieve standard simulation-based security for MPC.

To address this limitation, we apply an alternative method, called *collaborative sampling*, in which all participating parties jointly generate additive shares of the smudging noise. We consider both asymmetric two-party protocols between a client and a server, and symmetric multiparty protocols among mutually distrusting parties. For each setting, we present concrete protocol constructions based on collaborative sampling, explicitly define the corresponding ideal functionalities, and provide formal security proofs. We further provide concrete constructions and implementations of collaborative sampling protocols.

As an alternative perspective, we show that existing protocols, despite failing to achieve standard simulation-based security, still satisfy a weaker security notion called *liberal security* for approximate MPC protocols.

Keywords: Secure Approximate Computation, Homomorphic Encryption, CKKS, Multi-party Computation

1 Introduction

Homomorphic Encryption (HE) is a cryptographic primitive that enables computations to be performed directly on encrypted data without decryption. This property allows secure outsourcing of computation, enabling clients to leverage the computational power of an untrusted cloud while preserving data privacy. Due to its conceptual simplicity, broad applicability, and user-centric design, HE has emerged as one of the most promising privacy-enhancing technologies.

A primary use case of HE is in the design of two-party computation (2PC) protocols, in which one party holds the secret key while the other performs computations as an evaluator. This framework supports a wide range of secure outsourced computation scenarios, such as secure inference [15, 31, 50]. Beyond the two-party setting, several multi-party variants of HE, along with protocols built upon these primitives, have been actively proposed and studied in recent years (e.g., [16, 48, 52, 53, 54]). These HE-based Multi-Party Computation (MPC) protocols are particularly well-suited to cloud environments, such as federated learning [28, 36, 46], as they typically offer lower round complexity and reduced communication overhead than generic MPC protocols [22, 37, 38].

Although HE serves as a fundamental primitive for secure computation, the security of the underlying HE scheme does not naturally translate to the simulation-based security of the computation protocol.

Achieving standard MPC security requires additional cryptographic mechanisms beyond the straightforward application of HE. Fortunately, in the realm of exact HE (e.g., BFV [13], BGV [14]), methodologies for building secure MPC are well-established; for instance, combining IND-CPA security with circuit privacy [25, 30, 34] is sufficient to ensure simulation-based security of the 2PC protocol.

Meanwhile, among HE schemes, the CKKS scheme [18] has attracted significant attention for its support of approximate arithmetic, making it particularly suitable for applications involving real-number computations such as privacy-preserving machine learning [39, 43, 56]. A key distinguishing feature of CKKS compared to exact HE schemes is that it interprets noise, whether intentionally added for security or incurred through homomorphic operations, as an inherent part of the plaintext. Consequently, decrypting a CKKS ciphertext produces an approximate value, which inherently contains the accumulated error introduced throughout encryption and evaluation.

While this inherent nature plays a significant role in improving the efficiency of computations, it simultaneously introduces new security challenges. Notably, Li and Micciancio [44] demonstrated that revealing the decryption of a CKKS ciphertext without additional security-enhancing techniques can lead to key-recovery attacks, motivating the introduction of the IND-CPA^{D} security notion. Since then, several studies have actively explored security enhancements for CKKS from various perspectives. This includes extensive research on attacks [17, 33] and countermeasures [45, 47] related to the IND-CPA^{D} model, as well as the concept of circuit privacy within the context of approximate HE [41].

We argue, however, that analyzing these security elements in isolation (such as solely focusing on IND-CPA^{D} security or circuit privacy) does not yield a sufficient foundation for end-to-end protocol design. In fact, relying on these fragmented notions may lead to a false perception of security when constructing formally secure systems. Unfortunately, we observe that existing methods proposed to enhance the security of CKKS, while effective at mitigating specific vulnerabilities, fundamentally fail to translate into designing approximate computation protocols that achieve standard simulation-based MPC security.

1.1 Our Contributions

The inherent approximate nature of CKKS introduces unique challenges in the design of secure protocols. Extending well-known techniques (such as those used to translate exact HE schemes into secure protocols or noise flooding mechanisms designed to achieve IND-CPA^{D} security in CKKS) to end-to-end approximate MPC protocols is highly non-trivial. This is because, as these techniques typically rely on injecting additional noise, they directly alter the underlying plaintext values. Consequently, constructing formally secure protocols requires an integrated methodology that analyzes security and correctness (functionality) jointly, rather than treating them as independent properties.

To address these challenges, this work provides a comprehensive study of the security and functionality of approximate computation protocols, presenting the following contributions. First, we classify existing CKKS-based protocols and formally describe the ideal functionalities they actually realize. Contrary to folklore beliefs, we also demonstrate that a straightforward combination of existing CKKS security enhancement approaches is fundamentally insufficient to achieve standard simulation-based security. Through this, we show that most existing CKKS-based protocols in the literature are strictly insecure under standard cryptographic definitions.

Second, we propose a generic framework and methodology for designing approximate computation protocols that successfully achieve standard simulation-based security across representative HE deployment scenarios. We formally define the ideal functionalities for these protocols and provide rigorous security proofs, marking this as the first work to provide such formal treatments.

Finally, we investigate a relaxed, alternative security framework termed *liberal security*. While weaker than standard simulation-based security, we prove that existing CKKS-based protocols with independent noise-smudging methodologies formally satisfy this relaxed notion. This finding provides a theoretical justification for widely deployed protocols, showing that they are not completely insecure but rather offer a minimum security guarantee.

Overall, this work provides a comprehensive study of CKKS protocol designs, formally bridging the gap between their functionalities and actual security guarantees. By establishing both standard and liberal security frameworks, our results also introduce a practical trade-off between efficiency and security in

protocol design, thereby providing greater design flexibility depending on the target threat or security model.

1.2 Technical Overview

In this work, we examine fundamental security issues in the design of MPC protocols based on the CKKS scheme. We consider two main scenarios: asymmetric 2PC between a client and a server, and symmetric MPC among multiple parties.

Insecurity of Existing CKKS-based Protocols. We first demonstrate that the folklore belief that combining IND-CPA (or IND-CPA^{D}) security with circuit privacy or secure distributed decryption ensures simulation-based security of the MPC is fundamentally invalid in the setting of CKKS. Since a noise smudging technique is widely regarded as a standard countermeasure for achieving IND-CPA^{D} security [7, 45] and circuit privacy [41], existing CKKS-based 2PC protocols typically adopt noise smudging in both sides.

A similar strategy is employed in the multi-party setting. Recent works [42, 51] typically utilize the threshold CKKS scheme to support computation among multiple parties. In these protocols, the output can be obtained through distributed decryption involving decrypting parties. To construct secure distributed decryption, each participating party samples an independent smudging noise and adds it to their partial decryption share before publishing it.

However, we show that these approaches are insufficient to achieve standard simulation-based security. The core issue lies in the fact that, in CKKS, the smudging noise is not merely an auxiliary value but is tightly coupled with the protocol's functionality. In CKKS-based protocols, the final output, denoted as z, is typically derived as the sum of the exact computational result $\hat{z}$ and the smudging noise terms e_i sampled individually by each party (i.e., $z = \hat{z} + \sum_i e_i$) in both 2PC and MPC protocols. Crucially, because a corrupted party P_i knows its own noise contribution e_i, the simulator faces an impossible challenge while accounting for the adversary's known noise e_i: it must simulate an adversary's view that preserves the joint distribution between this view and the output of the ideal functionality indistinguishable from that of a real protocol. This implies that the adversary in the real world can derive a more precise approximation of the result (e.g., $z - e_i$) than what is allowed by the ideal functionality, thereby rendering the protocol insecure.

Secure Approximate MPC based on Collaborative Sampling. To overcome the fundamental limitations identified above, we apply an alternative technique termed *collaborative sampling* to construct the secure approximate computation protocol. Our analysis reveals a critical lesson: while the use of noise smudging is inevitable to address the approximate nature of CKKS, the standard approach of independent sampling fails because the smudging error is tightly coupled with the protocol's functionality. Specifically, as long as a subset of parties possesses non-negligible information regarding the smudging error embedded in the final output, constructing a valid simulator is infeasible.

To resolve this conflict, our proposed collaborative sampling mechanism shifts the noise generation from a local process to a joint computation. Instead of sampling noise terms individually, parties collaboratively generate additive shares of a global smudging noise. In this framework, each party holds only a random share that is statistically independent of the total accumulated noise. Therefore, while the final output z contains the necessary smudging error e for security (i.e., $z = \hat{z} + e$), no individual party gains knowledge of the actual value of e from their own smudging error e_i. This effectively decouples the adversary's view from the specific noise realization, thereby bridging the gap between the real execution and the ideal functionality.

We instantiate this approach by constructing new CKKS-based protocols for both two-party and multi-party settings. Unlike prior works that lacked formalism, we explicitly define the corresponding ideal functionalities and provide rigorous security proofs, demonstrating that our protocols achieve standard simulation-based security. We further provide concrete constructions and performance evaluations for collaborative sampling tailored to bounded uniform distributions. To realize collaborative noise sampling among participating parties, we design round-efficient random-bit sharing protocol rather than using interactive MPC protocols [21] as a fundamental building block.

Approximate MPC with Relaxed Security. From a security perspective, we also propose a relaxed variant, called *liberal security*, inspired by Feigenbaum et al. [27]. Unlike the standard definition, in which the simulator relies solely on the ideal functionality's output, the liberal definition grants the simulator access to auxiliary information. However, the original formulation in [27] imposed a restrictive assumption, including the requirement that the ideal functionality could be effectively simulated by the auxiliary information.

In this work, we redefine the notion of liberal security without this requirment, making it a more flexible and broadly applicable framework. Using this definition, we prove that existing CKKS-based protocols (which utilize party-wise noise smudging and fail to achieve standard security) formally satisfy this relaxed security notion. We provide the corresponding ideal functionalities and security proofs, offering a theoretical justification for widely-used, efficient protocols.

1.3 Related Works

In HE-based 2PC protocols, noise smudging (flooding) [30, 54] is a standard technique to mitigate threats to client privacy. In the context of approximate HE, since standard schemes remain vulnerable to key-recovery attacks [33, 44], calibrated noise smudging is employed to accommodate worst-case error growth [10,45]. Meanwhile, circuit privacy and sanitization techniques [11,25,34] have been explored to protect the evaluator's privacy, typically by randomizing the resulting ciphertext. However, it remains unclear how to seamlessly integrate these techniques to secure both client and server in approximate 2PC protocols.

Several variants extend CKKS to multi-party settings, including threshold CKKS [9, 53], multi-key CKKS [16,40], and multi-group CKKS [42]. Protocols built on these primitives [28,51] perform distributed decryption where each key owner publishes a partial decryption. While these works employ individual noise smudging to prevent leakage from partial shares, they lack explicit functionality definitions and formal security proofs, a gap that this work aims to address.

Threshold HE has been studied in both synchronous and asynchronous settings, depending on whether the set of decryptors is determined prior to the execution of the decryption protocol. In the asynchronous setting, [9] introduced a threshold HE scheme with a universal thresholdizer applicable to a broad class of cryptographic schemes. Subsequently, [26] proposed a stronger notion of threshold HE that captures security against adaptive adversaries in the asynchronous setting. They further presented a construction achieving this stronger security by combining key-homomorphic pseudorandom functions (PRFs).

In the synchronous setting, [51, 52] presented concrete constructions of threshold HE with practical parameter choices. However, these works neither formalized an ideal functionality nor provided a rigorous security proof for their proposed constructions. To highlight these limitations, [58] demonstrated a key-recovery attack against prior schemes. Furthermore, they proposed a secure threshold HE scheme in the synchronous model by applying the masking technique introduced in [23], together with a formal security proof.

2 Preliminaries

Notations. We use bold lower-case letters to denote column vectors. For an integer $n \in \mathbb{N}$, we write $[n] = \{1, 2, \ldots, n\}$. For a power-of-two integer N and an integer q, we denote by $R = \mathbb{Z}[X]/(X^N + 1)$ the ring of integers of the $2N$-th cyclotomic field and $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ the residue ring of R modulo q. We write $x \leftarrow \mathcal{D}$ to denote sampling x from a distribution $\mathcal{D}$, and $x \xleftarrow{\$} S$ to denote sampling (x uniformly at random from a finite set S. The statistical distance between two distributions $\mathcal{D}_1$ and $\mathcal{D}_2$ over a set S is defined as $\frac{1}{2} \cdot \sum_{x \in S} |\mathcal{D}_1(x) - \mathcal{D}_2(x)| \in [0, 1]$. We denote the uniform distribution over S by $\mathcal{U}(S)$ when S is finite. For a vector x in $\mathbb{R}^N$, the ℓ_∞-norm of x is defined as $\|x\|_\infty = \max_{i \in [N]}\{|x_i|\}$. Throughout the paper, we also use the notation $\|\cdot\|$ to denote the infinity norm for simplicity.

2.1 Smudging Distribution

In the context of decisional security problems, we aim to establish that a randomized ciphertext by applying the noise smudging technique is independent of the underlying secret information. Formally, it requires demonstrating that no efficient adversary can distinguish whether a randomized ciphertext includes private information. To prove this property, one typically analyzes the distance between two distributions: the distribution of randomized ciphertexts containing private information and that of randomized ciphertexts without it. Building on this framework, we define a smudging distribution as follows.

Definition 1. *(Informal) Let $E > 0$ be a constant. A distribution $\mathcal{D}$ over R is called an E-smudging distribution if $\mathcal{D}$ and $e + \mathcal{D}$ are statistically indistinguishable for any $e \in R$ such that $\|e\| \leq E$.*

The choice of the smudging distribution plays a crucial role, with the uniform distribution serving as a natural candidate. Specifically, if the distribution $\mathcal{D}$ is uniform over the range $[-E \cdot 2^{\lambda}, E \cdot 2^{\lambda})$, the statistical distance between the uniform distribution $\mathcal{D}$ and its shifted version (corresponding to an e such that $\|e\| \leq E$) is bounded by $2^{-\lambda}$ which implies that the distribution $\mathcal{D}$ is an E-smudging distribution.

2.2 The CKKS Scheme

Since Gentry's breakthrough [30], various HE schemes based on the (Ring) Learning with Errors (LWE/RLWE) assumption have been proposed, including BGV [14], BFV [13], FHEW/TFHE [19, 24], and CKKS [18]. Among these, the CKKS scheme has received attention due to its support of arithmetic for approximate numbers. In particular, the message space and the plaintext spaces are defined as $\mathbb{C}^{N/2}$ and $R = \mathbb{Z}[X]/(X^N + 1)$ for a power-of-two N, respectively.

• $\underline{\texttt{CKKS.Setup}(1^{\lambda})}$: For a security parameter λ, choose the ring degree N, ciphertext modulus $Q = \prod_{i=0}^{L} q_i$, special modulus P, key distribution χ_s and error distribution χ_e over R. Choose the gadget decomposition $h : R_Q \to R^k$ and gadget vector $\mathbf{g} \in R_Q^k$. Output the parameter set $\mathsf{pp} = (N, P, Q, \chi_s, \chi_e, h, \mathbf{g})$.

In CKKS, each ciphertext is labeled with a level $0 \leq \ell \leq L$, and a level-ℓ ciphertext has modulus $Q_\ell := \prod_{i=0}^{\ell} q_i$. We say that $h : R_Q \to R^k$ is a gadget decomposition corresponding to a gadget vector $\mathbf{g} \in R_Q^k$ if, for all $a \in R_Q$, $h(a) \in R^k$ is short (e.g., $\|h(a)\|_\infty$ is small) and satisfies $\langle h(a), \mathbf{g} \rangle = a \pmod{Q}$.

• $\underline{\texttt{CKKS.KeyGen}(\mathsf{pp})}$: Sample the secret key $\mathsf{sk} = s \leftarrow \chi_s$. Sample $a \xleftarrow{\$} R_{PQ}$ and $e \leftarrow \chi_e$, and set the public key as $\mathsf{pk} = (b, a) \in R_{PQ}^2$ where $b = -s \cdot a + e \pmod{PQ}$. Sample $\mathbf{r}_1 \xleftarrow{\$} R_{PQ}^k$ and $\mathbf{e} \leftarrow \chi_e^k$, and set the evaluation key as $\mathsf{evk} = (\mathbf{r}_0, \mathbf{r}_1) \in R_{PQ}^{k \times 2}$ where $\mathbf{r}_0 = -s \cdot \mathbf{r}_1 + \mathbf{e} + P \cdot s^2 \cdot \mathbf{g} \pmod{PQ}$. Return $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk})$.

• $\underline{\texttt{CKKS.Enc}(\mathsf{pk}; \mu)}$: Given the public key $\mathsf{pk} = (b, a)$ and a plaintext $\mu \in R$, sample $t \leftarrow \chi_s$ and $e_0, e_1 \leftarrow \chi_e$. Output the ciphertext $\mathsf{ct} = \left\lfloor \frac{1}{P}(t \cdot b + e_0, t \cdot a + e_1) \right\rceil + (\mu, 0) \pmod{Q}$.

• $\underline{\texttt{CKKS.Add}(\mathsf{ct}, \mathsf{ct}')}$: Given two ciphertexts $\mathsf{ct}, \mathsf{ct}' \in R_{Q_\ell}^2$ at level ℓ, output $\mathsf{ct}_{add} = \mathsf{ct} + \mathsf{ct}' \pmod{Q_\ell}$.

• $\underline{\texttt{CKKS.Mult}(\mathsf{evk}; \mathsf{ct}, \mathsf{ct}')}$: Given two ciphertexts $\mathsf{ct} = (c_0, c_1), \mathsf{ct}' = (c_0', c_1') \in R_Q^2$ at level ℓ and the relinearization key $\mathsf{evk} \in R_{PQ}^{d \times 2}$, compute $d_0 = c_0 \cdot c_0'$, $d_1 = c_0 \cdot c_1' + c_1 \cdot c_0'$, $d_2 = c_1 \cdot c_1' \pmod{Q_\ell}$ and output the ciphertext

$$\mathsf{ct}_{mul} = \left(d_0 + \left\lfloor \frac{1}{P} \langle h(d_2), \mathbf{r}_0 \rangle \right\rceil, d_1 + \left\lfloor \frac{1}{P} \langle h(d_2), \mathbf{r}_1 \rangle \right\rceil \right) \pmod{Q_\ell}.$$

• $\underline{\texttt{CKKS.Rescale}(\mathsf{ct})}$: Given a ciphertext $\mathsf{ct} = (c_0, c_1) \in R_{Q_\ell}^2$ at level ℓ, return $\mathsf{ct}' = \left(\left\lfloor q_\ell^{-1} \cdot c_0 \right\rceil, \left\lfloor q_\ell^{-1} \cdot c_1 \right\rceil \right) \in R_{Q_{\ell-1}}^2$.

• $\underline{\texttt{CKKS.Dec}(\mathsf{sk}; \mathsf{ct})}$: Given a ciphertext $\mathsf{ct} = (c_0, c_1) \in R_{Q_\ell}^2$ and the secret key $\mathsf{sk} = s$, return $\mu = c_0 + c_1 \cdot s \pmod{Q_\ell}$.

Correctness. CKKS is an approximate HE scheme, where the encryption and evaluation algorithms introduce small errors into the plaintext. As a result, the correctness of CKKS is defined in an approximate manner, by bounding the error accumulated during the evaluation of a circuit.

Definition 2 (Approximate Correctness [45]). *Let Σ be an HE scheme over a message space $\mathcal{M}$, and $\|\cdot\|$ be a norm defined on $\mathcal{M}$. Let $C : \mathcal{M}^k \to \mathcal{M}$ be a circuit with k input wires and let $\mathsf{Est} : \mathcal{L} \times \mathbb{R}_{\geq 0}^k \to \mathbb{R}_{\geq 0}$ be an efficiently computable function for a class of circuits $\mathcal{L}$. We say that Σ satisfies approximate correctness for the circuit C with respect to the error estimator Est if for all $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$, any messages $m_i \in \mathcal{M}$ such that $\|m_i\| \leq B_i$, and $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(m_i)$ for $1 \leq i \leq k$, the following holds:*

$$\|\mathtt{Dec}_{\mathsf{sk}}(\mathtt{Eval}_{\mathsf{evk}}(C, \{\mathsf{ct}_i\}_i)) - C(\{m_i\}_i)\| \leq \mathsf{Est}(C, B_1, \ldots, B_k)$$

The estimator Est typically computes either a provable worst-case bound or a heuristic average-case bound on the error, taking into account both the evaluation circuit C and the error bounds B_i associated with inputs [10, 20].

Security. We say that an approximate HE scheme is secure if it provides ciphertext indistinguishability under chosen plaintext attack ($\mathsf{IND\text{-}CPA}$). It is well known that CKKS is $\mathsf{IND\text{-}CPA}$ secure under the RLWE assumption and an additional circular security assumption. Li and Micciancio [44] noted that the traditional formulation of $\mathsf{IND\text{-}CPA}$ security may not capture the security of approximate HE schemes when the decryption results are shared with external parties. To address this, they introduced a new security notion, $\mathsf{IND\text{-}CPA^D}$ security, which gives an adversary access to a decryption oracle that decrypts ciphertexts derived from encrypting and evaluating its chosen messages. The formal definition of the $\mathsf{IND\text{-}CPA^D}$ security is provided in Appendix A.

Threshold CKKS. In standard (single-key) CKKS, full authority is concentrated in the secret key owner. To address this limitation, various multi-party variants of CKKS have been proposed, such as threshold CKKS [9, 51, 53], multi-key CKKS [16, 54], and multi-group CKKS [42]. As a typical example, we describe an n-out-of-n threshold CKKS scheme, with the t-out-of-n version provided in Appendix B. In this variant, parties collaboratively generate a shared public key and receive an additive share of the shared secret key as follows.

• $\underline{\mathtt{ThCKKS.Setup}(1^\lambda)}$: It is similar to $\mathtt{CKKS.Setup}(1^\lambda)$, but generates a common reference string $a \xleftarrow{\$} R_{PQ}$ and includes it in the public parameter pp.

• $\underline{\mathtt{ThCKKS.KeyGen}(\mathsf{pp})}$:

1. Each party P_i samples $\mathsf{sk}_i = s_i \leftarrow \chi_s$, $e_i \leftarrow \chi_e$ and publishes $b_i := -a \cdot s_i + e_i \pmod{PQ}$.
2. The public key is computed as $\mathsf{pk} := (b, a) \in R_{PQ}^2$ where $b = \sum_{i=1}^n b_i \pmod{PQ}$. Each party P_i samples $\mathbf{u}_i \xleftarrow{\$} R_{PQ}^k$ and $\mathbf{e}_{0,i}, \mathbf{e}_{1,i} \leftarrow \chi_e^k$, then publishes $(\mathbf{r}_{0,i}, \mathbf{r}_{1,i}) := (b \cdot \mathbf{u}_i + \mathbf{e}_{0,i}, a \cdot \mathbf{u}_i + s_i \cdot \mathbf{g} + \mathbf{e}_{1,i}) \pmod{PQ}$.
3. The evaluation key $\mathsf{evk} = (\mathbf{r}_0, \mathbf{r}_1) \in R_{PQ}^2$ is computed as $\sum_{i \in [n]} (\mathbf{r}_{0,i}, \mathbf{r}_{1,i}) \pmod{PQ}$.

At the end of the key generation, each party P_i obtains an additive share $s_i \in R$ of the secret key $\mathsf{sk} := \sum_{i=1}^n s_i$. In addition, all parties share the public key pk and evaluation key evk. It is also known that the shared public and evaluation keys can be generated non-interactively in a single round [42].

As a set of shared keys generated by the key generation protocol has the same structure as those in the single-key CKKS, threshold CKKS can use the same encoding/decoding, encryption, and evaluation algorithms. The decryption of the ciphertext can be carried out in a distributed manner as discussed in [42, 51]. During the decryption protocol, each party partially decrypts the ciphertext using its partial secret key and publishes it after adding a smudging noise [3, 9]. The decrypted result is then obtained by aggregating all partial decryptions.

• $\underline{\mathtt{ThCKKS.DistDec}(\{\mathsf{sk}_i\}_{i \in [n]}, \mathsf{ct})}$: The distributed decryption protocol proceeds as follows for a given ciphertext $\mathsf{ct} = (c_0, c_1) \in R_{Q_\ell}^2$:

1. Each party P_i samples $e_i \leftarrow \mathcal{D}$ where $\mathcal{D}$ denotes the smudging distribution, then publishes $\mu_i := c_1 \cdot s_i + e_i \pmod{Q_\ell}$.
2. All parties compute and output $m := c_0 + \sum_{i \in [n]} \mu_i \pmod{Q_\ell}$.

2.3 Secure Multi-party Computation

Let n be a number of parties and let $f : (\{0,1\}^*)^n \to (\{0,1\}^*)^n$ be a (n-ary) probabilistic polynomial-time (PPT) functionality. We denote $\boldsymbol{x} = (x_1, \ldots, x_n)$ and $f(\boldsymbol{x}) = (f_1(\boldsymbol{x}), \ldots, f_n(\boldsymbol{x}))$, where x_i and f_i correspond to the party P_i. For an MPC protocol Π that computes the functionality f on input $\boldsymbol{x}$, the view of P_i, denoted by $\mathsf{view}_i^{\Pi}(\boldsymbol{x})$, is defined as the tuple of its input, randomness, and all messages received from other parties. We denote the output of P_i from the execution of Π on input $\boldsymbol{x}$ by $\mathsf{output}_i^{\Pi}(\boldsymbol{x})$, and $\mathsf{output}^{\Pi}(\boldsymbol{x}) = (\mathsf{output}_1^{\Pi}(\boldsymbol{x}), \ldots, \mathsf{output}_n^{\Pi}(\boldsymbol{x}))$. We assume that the protocol proceeds synchronously, where in each round, each party sends a message based on its input, random input, and previously received messages. At the end of each round, parties may terminate and output some value based on their entire view, including their input, random input, and received messages. The definition of the secure MPC is as follows.

Definition 3 (Secure Multi-party Computation [32]). *Let Π be an n-party protocol for computing a functionality $f = (f_1, \ldots, f_n)$. For a set of indices $A \subseteq \{1, \ldots, n\}$, we denote $f_A(\boldsymbol{x}) = (f_i(\boldsymbol{x}))_{i \in A}$ and $\mathsf{view}_A^{\Pi}(\boldsymbol{x}) = (\mathsf{view}_i^{\Pi}(\boldsymbol{x}))_{i \in A}$. We say that Π securely realizes f under the presence of semi-honest adversaries if there exists PPT algorithms $\mathtt{Sim}^{\Pi}$ such that for every $A \subseteq \{1, \ldots, n\}$ and $\boldsymbol{x}$, the following holds:*

$$\{(\mathtt{Sim}^{\Pi}(A, (x_i)_{i \in A}, f_A(\boldsymbol{x})), f(\boldsymbol{x}))\} \approx_c \{(\mathsf{view}_A^{\Pi}(\boldsymbol{x}), \mathsf{output}^{\Pi}(\boldsymbol{x}))\}.$$

The above privacy requirement asserts that the *joint distribution* of the simulator's output and the outputs of functionality should be indistinguishable from the view of the corrupted party and the outputs of parties. This requirement is particularly meaningful when the functionality is probabilistic, as it guarantees that the adversary does not obtain any additional information about the output. Throughout the paper, we refer to this definition as standard simulation-based security.

3 Insecurity of Existing CKKS-based Protocols

Although HE serves as a foundational primitive for secure computation, the security of the underlying encryption scheme does not inherently guarantee the security of the resulting protocol. Fortunately, methodologies for building secure computation protocols from exact HE schemes have been well established; for example, noise smudging is a widely adopted technique to ensure secure distributed decryption [21, 57].

However, it remains unclear whether these approaches can be directly extended to CKKS due to its inherent approximate nature. In particular, conventional security-enhancing techniques, such as noise smudging, alter the underlying plaintext values in the CKKS setting. Consequently, designing CKKS-based protocols presents a unique challenge: one must simultaneously address not only security but also the functionality (i.e., correctness) of the protocol. While prior studies on CKKS-based protocols have identified and partially addressed these issues, none have explicitly defined the ideal functionality or successfully achieved simulation-based security for the entire MPC protocol.

In this section, we demonstrate that existing CKKS-based protocols are inherently insecure under the standard simulation-based security definition. To this end, we explicitly define the ideal functionalities of existing approaches and formally prove that these protocols fail to securely realize them.

3.1 System Models

We focus on two primary deployment scenarios for HE: (1) asymmetric 2PC between a client and a server, and (2) symmetric MPC among multiple parties.

Asymmetric Two-party Protocol. Asymmetric two-party protocols constitute one of the most common applications of HE. In this setting, the client acts as the key owner and the server as the evaluator, collaboratively computing a public circuit C on their respective private inputs.

1. The client generates a key pair, encrypts its private input, and sends the ciphertext and the evaluation key to the server.

2. The server homomorphically evaluates the circuit on the received ciphertext and its private input, and sends the resulting ciphertext back to the client.
3. The client decrypts the output ciphertext to obtain the final result and shares it with the server.

This baseline solution raises several security concerns. First, the decrypted value of a CKKS ciphertext may leak more information than the client intends. For instance, if the client shares the exact decrypted value of the output ciphertext with the server, the server can recover the client's secret key [44]. On the other hand, the server also needs to ensure its input privacy, which is often referred to as circuit privacy in the HE context. A standard approach to achieving circuit privacy is to randomize the output ciphertext by adding an encryption of zero and an additional smudging noise, thereby removing extraneous information beyond the underlying plaintext [41]. Therefore, the two aforementioned issues must be considered together for constructing a secure CKKS-based 2PC protocol.

Symmetric Multi-party Protocol. To support the symmetric MPC setting, several multi-party variants of the CKKS scheme have been proposed [16,40,42,51,53]. Building on these primitives, studies such as [28,51,53] have investigated privacy-preserving MPC protocols using threshold CKKS, demonstrating the feasibility of secure computation over approximate values. These primitives have found widespread adoption in practice, most notably in the field of privacy-preserving federated learning [28,36,46].

Typically, threshold HE-based protocols follow this following pipeline:

1. Execute a key-generation protocol, such that each party obtains a common public key and a share of the corresponding secret key.
2. Each party encrypts its private input using the public key and broadcasts the ciphertext. Subsequently, the parties homomorphically evaluate the circuit on ciphertexts.
3. The parties collaboratively decrypt the output ciphertext and obtain the result.

Typically, the final decryption is performed by letting each party compute a partial decryption using their respective secret share and broadcast it [16, 53]. The final result is then recovered by aggregating these partial decryptions.

3.2 Impossibility of Standard Security with Noise Smudging

While prior works have attempted to mitigate potential security vulnerabilities in CKKS-based protocols through various defense strategies [2, 7, 45, 47], they do not explicitly define the ideal functionality of the end-to-end protocol. Consequently, their security arguments remain incomplete. In this section, we demonstrate that existing approaches not only suffer from a lack of formalism, but are also fundamentally insufficient to achieve standard simulation-based MPC security.

Two-party Case. At first glance, $\mathsf{IND\text{-}CPA}$ (or $\mathsf{IND\text{-}CPA^D}$) security and circuit privacy seem to be the two essential requirements to achieve secure approximate 2PC protocols. Indeed, in exact computation, an HE scheme satisfying both IND-CPA security and circuit privacy is sufficient to guarantee simulation-based security of the resulting 2PC protocol. However, we show that this intuition does not apply to CKKS. Contrary to folklore belief, a straightforward combination of the $\mathsf{IND\text{-}CPA^D}$ security of CKKS and circuit privacy is insufficient for secure approximate 2PC protocols, ultimately failing to satisfy the standard security.

Specifically, consider a 2PC protocol in which the client and the server independently employ noise smudging to achieve $\mathsf{IND\text{-}CPA^D}$ security and circuit privacy, respectively, as illustrated in Fig 1. Throughout this subsection, we omit the prefix $\mathtt{CKKS}$ from the CKKS algorithms when it is clear from the context. Let $\hat{z}$ denote the result of the exact evaluation, and let e_1 and e_2 denote the smudging noise sampled by the client and the server, respectively. Then, the output of the protocol z_{out} is statistically close to $\hat{z} + e_1 + e_2$ since e_1 and e_2 are sampled from the smudging distribution. Accordingly, the functionality of this protocol can be represented as in Fig. 2.

We show that the 2PC protocol Π_{Prev2PC}, which realizes the functionality $\mathcal{F}_{\mathrm{Prev2PC}}$, does not satisfy the standard simulation-based security in general. Specifically, the protocol is insecure even for the simple circuit $C(x, y) = x + y$.

Protocol Π_{Prev2PC}

1. P_1 generates a key pair $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and encrypts $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$. P_1 sends $(\mathsf{pk}, \mathsf{ct}_{\mathsf{in}})$ to P_2.
2. P_2 computes $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}, y)$, $\mathsf{ct}_{\mathsf{zero}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$, and samples $e_2 \leftarrow \mathcal{D}$. P_2 computes $\mathsf{ct}'_{\mathsf{out}} := \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (e_2, 0)$, and sends it to P_1.
3. P_1 samples $e_1 \leftarrow \mathcal{D}$ and computes $z_{\mathsf{out}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}'_{\mathsf{out}}) + e_1$.
   P_1 sends z_{out} to P_2, and both parties output z_{out}.

Fig. 1. Existing CKKS-based approximate 2PC protocol.

Functionality $\mathcal{F}_{\mathrm{Prev2PC}}$

Setup:
Let C be an evaluation circuit and $\mathcal{D}$ be an error distribution. The functionality receives x from P_1 (the client) and y from P_2 (the server).

1. Compute $\hat{z} := C(x, y)$.
2. Sample $e_1, e_2 \leftarrow \mathcal{D}$ and let $z := \hat{z} + e_1 + e_2$.
3. Output z to both P_1 and P_2.

Fig. 2. Functionality of the existing approximate 2PC.

Theorem 1. *Let $\Sigma = (\mathtt{KeyGen}, \mathtt{Enc}, \mathtt{Dec}, \mathtt{Eval})$ be the CKKS scheme with security parameter λ. Let $\Pi_{Prev2PC}$ be the protocol described in Fig. 1 for evaluating the circuit $C(x, y) = x+y$, and let the smudging distribution $\mathcal{D}$ be the uniform distribution. Then, this protocol fails to achieve standard simulation-based security.*

Proof. Suppose that there exists a PPT simulator $\mathtt{Sim}^{\Pi_{\mathrm{Prev2PC}}}$ satisfying standard simulation-based security for a corrupted client P_1. Since the client's view contains its locally sampled smudging noise, projecting the simulated view onto this component yields a PPT algorithm, which we also denote by $\mathtt{Sim}^{\Pi_{\mathrm{Prev2PC}}}$, that simulates this noise jointly with the ideal output.

Let $\mathcal{D} = \mathcal{U}\{-B, \ldots, B\}$, where $B = B(\lambda) \geq 1$ has bit-length polynomial of λ. Let us fix $x = 0$ and consider the two honest-party inputs $y_0 = 0$ and $y_1 = 2B + 1$. Let e_1, e_2, e'_1, e'_2 be independent samples from $\mathcal{D}$.

By the smudging property, the HE approximation error can be ignored with negligible statistical cost, even jointly with the client's noise, since the independently sampled honest-party noise smudges this error conditional on the client's noise. Therefore, the simulator $\mathtt{Sim}^{\Pi_{\mathrm{Prev2PC}}}$ must satisfy

$$\left(\mathtt{Sim}^{\Pi_{\mathrm{Prev2PC}}}(1^\lambda, 0, Z_b), Z_b\right) \approx_c (e_1, y_b + e_1 + e_2), \tag{1}$$

for $b \in \{0, 1\}$ where $Z_b := y_b + e'_1 + e'_2$ is the ideal output. We denote the left-hand side ideal distribution as I_b, and right-hand side real distribution as R_b.

Now, consider the polynomial-time distinguisher

$$T(v, z) := \mathbf{1}\{z - v \leq B\}.$$

In the real distributions, subtracting the client's noise gives $z - v = y_b + e_2$. Therefore, $\Pr[T(R_0) = 1] = 1$, and $\Pr[T(R_1) = 1] = 0$ because $e_2 \leq B$, whereas $y_1 + e_2 \geq B + 1$. This implies that the distinguishing gap of T is 1.

However, we show that any simulator cannot reproduce this gap. Note that I_b is a randomized process over Z_b, therefore by data processing inequality $\Delta(I_0, I_1) \leq \Delta(Z_0, Z_1)$. To calculate the latter distance, let $p(t) := \Pr[e'_1 + e'_2 = t]$. Since e'_1, e'_2 are independent and uniform over $\{-B, \ldots, B\}$,

$$p(t) = \begin{cases} \dfrac{2B + 1 - |t|}{(2B + 1)^2}, & |t| \leq 2B, \\ 0, & \text{otherwise.} \end{cases}$$

The support of Z_0 and Z_1 overlaps in $\{1, \ldots, 2B\}$. Hence,

$$\begin{aligned} 1 - \Delta(Z_0, Z_1) &= \sum_{t=1}^{2B} \min\{p(t), p(t-(2B+1))\} \\ &= \frac{1}{(2B+1)^2} \sum_{t=1}^{2B} \min\{2B+1-t, t\} = \frac{B(B+1)}{(2B+1)^2} \geq \frac{2}{9}. \end{aligned}$$

The last inequality holds for every integer $B \geq 1$, since $\frac{B(B+1)}{(2B+1)^2} - \frac{2}{9} = \frac{9B(B+1)-2(2B+1)^2}{9(2B+1)^2} = \frac{(B-1)(B+2)}{9(2B+1)^2} \geq 0$. For each $b \in \{0,1\}$, let q_b denote the probability that T accepts the simulated result when the honest party's input is y_b. Then, we can write

$$\begin{aligned} q_b &:= \Pr[T(I_b) = 1] \\ &= \Pr\left[Z_b - \mathtt{Sim}^{\Pi_{\mathrm{Prev2PC}}}(1^\lambda, 0, Z_b) \leq B\right]. \end{aligned}$$

The probability is taken over the ideal noises e'_1, e'_2 and the simulator's internal randomness. By the defining property of total variation distance, the difference in the probabilities of any event is bounded by the distance between the two distributions. Applying this to the acceptance event of T, we obtain $|q_0 - q_1| \leq \Delta(I_0, I_1) \leq \frac{7}{9}$.

On the other hand, T accepts R_0 with probability 1 and R_1 with probability 0. Thus, simulation would require both $1-q_0$ and q_1 to be negligible. It is impossible because $(1-q_0)+q_1 \geq 1-|q_0-q_1| \geq \frac{2}{9}$. This is a contradiction, and therefore no such simulator exists. □

Remark 1. One might consider modifying the protocol so that the client and the server sample smudging noise from different distributions. Nevertheless, such a modification can provide security for at most one party; namely the party whose smudging error is exponentially larger than that of the other party.

Remark 2. We note that the existing computation protocols based on exact HE can be shown to satisfy standard simulation-based security. This property is derived from the fact that, in the exact HE setting, the outputs of both the real protocol and the ideal functionality are independent of both smudging errors e_1 and e_2. As a result, even if the simulator is provided with the smudging noise associated with the adversarial party, it cannot derive information that is more accurate than the output generated by the ideal functionality.

Remark 3. The analysis becomes considerably simpler in the restricted setting where the server holds no private input and merely evaluates a public circuit on the client's ciphertext. In this case, achieving $\mathsf{IND\text{-}CPA}^{\mathsf{D}}$ by applying noise smudging in the client-side is sufficient to guarantee the standard security. We present the corresponding functionality, protocol, and security proof in Appendix C. However, such a protocol has limited cryptographic significance. Since the server acts solely as a computational resource without contributing any private data or randomness, the resulting setting corresponds to secure computation delegation rather than genuine 2PC. Consequently, both its theoretical importance and practical applications are severely limited.

Multi-party Case. A similar obstacle arises in the multi-party setting. In prior works [42, 51], the distributed decryption procedure relies on noise smudging to prevent leakage of secret key shares: each party samples a smudging noise term, adds it to their partial decryption, and broadcasts the result, as illustrated in Fig. 3. Consequently, the final output is in the form of $z = \hat{z} + \sum_i e_i$, where $\hat{z}$ denotes the exact computation result and e_i denotes the smudging noise sampled by party P_i. Throughout this subsection, we omit the prefix `ThCKKS` from the CKKS algorithms when it is clear from the context. The ideal functionality $\mathcal{F}_{\mathrm{PrevMPC}}$ of this approximate MPC protocol is shown in Fig. 4.

The critical issue is that the adversary's view includes the smudging errors sampled by the corrupted parties. This imposes an impossible task on the simulator as similar as proof of Thm. 1: given only the final output z, the simulator must generate a view containing the corrupted parties' smudging errors that

Protocol Π_{PrevMPC}

1. All parties generate $(\mathsf{sk}_i, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. Each party P_i encrypts $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$ and broadcasts it.
3. All parties compute $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$.
   We denote $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}})$.
4. Each party P_i samples $e_i \leftarrow \mathcal{D}$, computes $u_i = a_{\mathsf{out}} \cdot s_i + e_i \pmod{q_{\mathsf{out}}}$ and broadcasts it.
5. All parties output $z_{\mathsf{out}} = b_{\mathsf{out}} + \sum_{i=1}^{n} u_i \pmod{q_{\mathsf{out}}}$.

Fig. 3. Existing CKKS-based approximate MPC protocol.

Functionality $\mathcal{F}_{\mathrm{PrevMPC}}$

Setup:
Let C be an evaluation circuit and $\mathcal{D}$ be an error distribution. The functionality receives x_i from n parties P_i $(1 \le i \le n)$.

1. Compute $\hat{z} = C(x_1, \ldots, x_n)$.
2. For $1 \le i \le n$, sample $e_i \leftarrow \mathcal{D}$ and let $z := \hat{z} + \sum_{i=1}^{n} e_i$.
3. Output z to all parties.

Fig. 4. Functionality of the existing approximate MPC.

is consistent with z. If such a simulation were possible, then the simulator could derive a more precise approximation of $\hat{z}$ than what is revealed by the ideal functionality itself, leading to a contradiction.

The key lesson from this section is that while the noise smudging technique is inevitable for addressing the inherent approximate nature and potential security vulnerabilities of CKKS, the straightforward approach, where each party simply samples their own smudging noise, is insufficient. The fundamental reason is that the smudging error is tightly coupled with the protocol's functionality. Consequently, if a subset of parties possesses non-negligible information regarding the smudging error contained in the final protocol output, constructing a valid simulator becomes infeasible.

4 Constructing Secure Approximate Computation Protocols

As discussed in the previous section, existing formulations of $\mathsf{IND\text{-}CPA}^{\mathsf{D}}$ security and circuit privacy for CKKS do not adequately capture the security requirements of CKKS-based approximate MPC protocols. A key insight from our analysis is that security cannot be achieved if any party retains non-negligible information about the smudging error, as no simulator can reconstruct the view of that party using only the ideal functionality. To address this limitation, we introduce a new technique, called *collaborative error sampling*, in which all participating parties jointly generate additive shares of the smudging noise. Based on this error sampling method, we explicitly define an ideal functionality for approximate computation and present CKKS-based two-party and multiparty protocols that securely realize this functionality. Consequently, the resulting protocols achieve standard simulation-based security in both two-party and multiparty settings.

4.1 Approximate 2PC with Standard Security

To construct a secure 2PC protocol, we modify the error sampling procedure to prevent any information leakage about the smudging error. Instead of each party sampling its own error independently, the parties collaboratively generate the smudging error. As a result, neither party obtains any partial knowledge of the smudging error, while they can still compute the final output.

To this end, we introduce the collaborative error sampling functionality $\mathcal{F}_{\text{2-ColSamp}}$ (illustrated in Fig. 5), which generates additive shares of the smudging error. In this two-party collaborative sampling

scheme, each individual share r_i is statistically indistinguishable from a uniformly sampled element from $R_{q_{\mathsf{out}}}$ where q_{out} denotes the ciphertext modulus of the output.

Functionality $\mathcal{F}_{\text{2-ColSamp}}$

Setup: Let $\mathcal{D}$ be a smudging error distribution over R and q_{out} denotes the modulus of output ciphertext.

1. Sample $e \leftarrow \mathcal{D}$.
2. Sample $r_1 \xleftarrow{\$} R_{q_{\mathsf{out}}}$ and compute $r_2 := -r_1 + e \pmod{q_{\mathsf{out}}}$.
3. Output r_i to P_i for $i = 1, 2$.

Fig. 5. Functionality of two-party collaborative sampling.

Since the client and the server share the additive shares of the smudging error e, the protocol output z can be expressed as $\hat{z} + e$, for the exact value of the circuit evaluation $\hat{z}$. Accordingly, the functionality $\mathcal{F}_{\text{New2PC}}$ of our approximate 2PC protocol is specified in Fig. 6 and the protocol realizing $\mathcal{F}_{\text{New2PC}}$ is presented in Fig. 7. We assume that the two parties P_1 (the client) and P_2 (the server) obtain the additive shares of the smudging noise from $\mathcal{F}_{\text{2-ColSamp}}$ during the offline phase. Throughout this subsection, we omit the prefix `CKKS` from the CKKS algorithms when it is clear from the context.

Functionality $\mathcal{F}_{\text{New2PC}}$

Setup:
Let C be an evaluation circuit and $\mathcal{D}$ be an error distribution. The functionality receives x from P_1 (the client) and y from P_2 (the server).

1. Compute $\hat{z} := C(x, y)$.
2. Sample $e \leftarrow \mathcal{D}$ and let $z := \hat{z} + e$.
3. Output z to both P_1 and P_2.

Fig. 6. Functionality of the secure approximate 2PC.

Protocol Π_{New2PC}

Offline:
P_1 and P_2 call $\mathcal{F}_{\text{2-ColSamp}}$ and each party P_i obtains r_i for $i = 1, 2$.

Online:

1. P_1 generates a key pair $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$. P_1 sends $(\mathsf{pk}, \mathsf{ct}_{\mathsf{in}})$ to P_2.
2. P_2 computes $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}, y)$ and $\mathsf{ct}_{\mathsf{zero}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$. P_2 computes $\mathsf{ct}'_{\mathsf{out}} := \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (r_2, 0)$, and sends it to P_1.
3. P_1 computes $z_{\mathsf{out}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}'_{\mathsf{out}} + (r_1, 0))$.
4. P_1 sends z_{out} to P_2. Both P_1 and P_2 output z_{out}.

Fig. 7. CKKS-based secure approximate 2PC protocol.

In the following, we show that our 2PC protocol Π_{New2PC} securely realizes the functionality for approximate computation. Due to the functionality of collaborative sampling, each value r_i in P_i's view is independent of the smudging error $e \leftarrow \mathcal{D}$. Recall that in the aforementioned (insecure) protocol, described in Section 3.2, the output is of the form $z_{\mathsf{out}} = z_{\mathsf{dec}} + e_1 + e_2$, where z_{dec} is the decryption result

of $\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}}$ and each $e_i \leftarrow \mathcal{D}$ is sampled from P_i for $i = 1, 2$. We note that $z_{\mathsf{dec}} + e_1 + e_2$ is statistically indistinguishable with the $\hat{z} + e_1 + e_2$ since e_i's are sampled from the smudging distribution. Therefore, each party can derive a more accurate approximation, namely $\hat{z} + e_1$ (or $\hat{z} + e_2$), than what the ideal functionality provides. In contrast, in the Π_{New2PC} protocol, the two parties collaboratively sample the error and the single smudging error e is added to the resulting ciphertext. As a consequence, both parties learn only the final output $z_{\mathsf{out}} = z_{\mathsf{dec}} + e \approx_s \hat{z} + e$ and obtain no additional information about the exact computation value $\hat{z}$. This key distinction ensures that the protocol Π_{New2PC} satisfies simulation-based security in the standard sense.

Theorem 2. *For plaintexts x, y and a circuit C, suppose that $\|x\| \le B_1$ and $\|y\| \le B_2$ for some $B_1, B_2 > 0$. Let $B_0 > 0$ be an upper bound of public-key encryption error, i.e., $\Pr[\|\mathsf{Dec}_{\mathsf{sk}}(\mathsf{Enc}_{\mathsf{pk}}(0))\| \le B_0]$ with an overwhelming probability. Let $E := \mathsf{Est}(C, B_1, B_2) + B_0$. If $\mathtt{CKKS}$ is $\mathsf{IND\text{-}CPA}$ secure and $\mathcal{D}$ is an E-smudging distribution, then the 2PC protocol Π_{New2PC} securely realizes the functionality $\mathcal{F}_{New2PC}$ in the $\mathcal{F}_{2\text{-ColSamp}}$-hybrid model.*

Proof. We first show the correctness of our protocols. Let $\hat{z} := C(x, y)$ and $z_{\mathsf{dec}} := \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}})$. Except with probability $\epsilon_{\mathsf{corr}}(\lambda)$, there exists $e_{\mathsf{out}} \in R$ such that

$$z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}} \quad \text{and} \quad \|e_{\mathsf{out}}\| \le E.$$

Therefore, the output can be written as $z_{\mathsf{out}} = \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}'_{\mathsf{out}} + (r_1, 0)) = \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (r_2, 0) + (r_1, 0)) = \hat{z} + e_{\mathsf{out}} + e$ for some $e \leftarrow \mathcal{D}$. Thus, the output distribution is statistically indistinguishable from the distribution of the output $\hat{z} + e$ of the functionality $\mathcal{F}_{\mathrm{New2PC}}$ since $\mathcal{D}$ is an E-smudging distribution.

To prove the security of Π_{New2PC}, we separately consider a corrupted client P_1 and a corrupted server P_2, using a sequence of hybrid arguments for each corruption case. For convenience, we use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted party's view and the protocol output.

We first consider a corrupted client P_1. Since both parties output the same value, we identify the output vector $(z_{\mathsf{out}}, z_{\mathsf{out}})$ with the common output z_{out}. The simulator $\mathtt{Sim}^1_{\mathrm{New2PC}}(x, z)$ for P_1's view is as follows.

1. Generate $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathsf{Enc}_{\mathsf{pk}}(x)$.
2. Sample $r_1 \xleftarrow{\$} R_{q_{\mathsf{out}}}$ and let $u' := z - r_1$.
3. Sample $a \xleftarrow{\$} R_{q_{\mathsf{out}}}$ and let $\mathsf{ct}'_{\mathsf{out}} := (u' - as, a)$.
4. Output $(x, r_1, \mathsf{sk}, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}'_{\mathsf{out}})$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world scenario. As mentioned in the correctness proof, the decrypted result $z_{\mathsf{dec}} := \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}})$ in the real protocol is $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ with $\|e_{\mathsf{out}}\| \le E$, and the output can be written as $z_{\mathsf{out}} = z_{\mathsf{dec}} + e = \hat{z} + e_{\mathsf{out}} + e$. The joint distribution of P_1's view and the protocol output is as follows:

$$\begin{aligned}
&\{(\mathsf{view}_1^{\Pi_{\mathrm{New2PC}}}(x, y), \mathsf{output}^{\Pi_{\mathrm{New2PC}}}(x, y))\} \\
\equiv &\{((x, r_1, \mathsf{sk}, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}'_{\mathsf{out}}), \hat{z} + e_{\mathsf{out}} + e) \mid (\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \\
&\mathsf{ct}_{\mathsf{in}} \leftarrow \mathsf{Enc}_{\mathsf{pk}}(x), \quad \mathsf{ct}_{\mathsf{out}} \leftarrow \mathsf{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}, y), \quad \mathsf{ct}_{\mathsf{zero}} \leftarrow \mathsf{Enc}_{\mathsf{pk}}(0), \\
&e \leftarrow \mathcal{D}, \quad r_1 \xleftarrow{\$} R_{q_{\mathsf{out}}}, \quad r_2 := e - r_1, \quad \mathsf{ct}'_{\mathsf{out}} := \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (r_2, 0)\}.
\end{aligned}$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that it replaces the effective error $e_{\mathsf{out}} + e$ with e. More precisely, it samples $e \leftarrow \mathcal{D}$ and defines $\widetilde{r}_2 := e - e_{\mathsf{out}} - r_1 \pmod{q_{\mathsf{out}}}$. It then computes $\mathsf{ct}'_{\mathsf{out}} := \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (\widetilde{r}_2, 0)$ and sets the output to $z_1 := \hat{z} + e$. By approximate correctness, $\|e_{\mathsf{out}}\| \le E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

Before proceeding to the next hybrid, let $\mathsf{ct}_{\mathsf{out}} = (b_{\mathsf{out}}, a_{\mathsf{out}})$ and $\mathsf{ct}_{\mathsf{zero}} = (b_{\mathsf{zero}}, a_{\mathsf{zero}})$. Since $(b_{\mathsf{out}} + b_{\mathsf{zero}}) + (a_{\mathsf{out}} + a_{\mathsf{zero}})s = \hat{z} + e_{\mathsf{out}}$, we have $b_{\mathsf{out}} + b_{\mathsf{zero}} = \hat{z} + e_{\mathsf{out}} - (a_{\mathsf{out}} + a_{\mathsf{zero}})s$. Therefore, $\mathsf{ct}'_{\mathsf{out}} = \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (e - e_{\mathsf{out}} - r_1, 0) = \big(\hat{z} + e - r_1 - (a_{\mathsf{out}} + a_{\mathsf{zero}})s, a_{\mathsf{out}} + a_{\mathsf{zero}}\big)$.

$\mathbf{Hyb}_2$: This hybrid is identical to $\mathbf{Hyb}_1$, except that a_{zero} of the fresh zero encryption $\mathsf{ct}_{\mathsf{zero}} = (b_{\mathsf{zero}}, a_{\mathsf{zero}}) \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ is replaced by an independently sampled $\widetilde{a}_{\mathsf{zero}} \xleftarrow{\$} R_{q_{\mathsf{out}}}$. Since a_{zero} is an RLWE sample with fresh short secret and error, it is computationally indistinguishable from uniform under the decisional RLWE assumption. The received ciphertext is accordingly generated as $\mathsf{ct}'_{\mathsf{out}} := \big(\hat{z} + e - r_1 - (a_{\mathsf{out}} + \widetilde{a}_{\mathsf{zero}})s,\ a_{\mathsf{out}} + \widetilde{a}_{\mathsf{zero}}\big)$, which satisfies $\mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}'_{\mathsf{out}} + (r_1, 0)) = \hat{z} + e$. Hence, $\mathbf{Hyb}_1 \approx_c \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: Since $\widetilde{a}_{\mathsf{zero}}$ is uniform over $R_{q_{\mathsf{out}}}$, the value $a := a_{\mathsf{out}} + \widetilde{a}_{\mathsf{zero}}$ is also uniform over $R_{q_{\mathsf{out}}}$, even conditioned on a_{out}. Therefore, we rewrite the received ciphertext as $\mathsf{ct}'_{\mathsf{out}} = \big(\hat{z} + e - r_1 - as, a\big)$, where $a \xleftarrow{\$} R_{q_{\mathsf{out}}}$. Thus, $\mathbf{Hyb}_2 \equiv \mathbf{Hyb}_3$.

$\mathbf{Hyb}_4$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{New2PC}}(x, y)$ instead of explicitly computing $\hat{z} = C(x, y)$ and sampling $e \leftarrow \mathcal{D}$. It then defines $u' := z - r_1$ and generates the received ciphertext as $\mathsf{ct}'_{\mathsf{out}} := (u' - as, a)$, where $a \xleftarrow{\$} R_{q_{\mathsf{out}}}$. By the definition of $\mathcal{F}_{\mathrm{New2PC}}$, its output z is distributed as $z = \hat{z} + e$, where $e \leftarrow \mathcal{D}$. Therefore, $\mathbf{Hyb}_3 \equiv \mathbf{Hyb}_4$. Note that $\mathbf{Hyb}_4$ corresponds to the simulator $\mathtt{Sim}^1_{\mathrm{New2PC}}(x, z)$.

Next, we construct the simulator $\mathtt{Sim}^2{}_{\mathrm{New2PC}}(y, z)$ to simulate P_2's view in the protocol Π_{New2PC} as follows:

1. Generate $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
2. Sample $r_2 \leftarrow R_{q_{\mathsf{out}}}$.
3. Output $(y, r_2, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}})$.

We construct the following hybrids:

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world scenario. The joint distribution of P_2's view and the protocol output is as follows:

$$
\begin{aligned}
&\{\mathsf{view}_2^{\Pi_{\mathrm{New2PC}}}(x, y), \mathsf{output}^{\Pi_{\mathrm{New2PC}}}(x, y)\} \\
\equiv &\{(y, r_2, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}), \hat{z} + e_{\mathsf{out}} + e \mid (\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \\
&(r_1, r_2) \leftarrow \mathcal{F}_{\text{2-ColSamp}}, \mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)\}
\end{aligned}
$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that it samples $r_2 \xleftarrow{\$} R_{q_{\mathsf{out}}}$ independently instead of obtaining it from $\mathcal{F}_{\text{2-ColSamp}}$. From the collaborative sampling functionality, $\mathbf{Hyb}_0 \equiv \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: In this hybrid, it computes the protocol output as $\hat{z} + e$, which was $\hat{z} + e_{\mathsf{out}} + e$ in the previous hybrid. By approximate correctness, $\|e_{\mathsf{out}}\| \leq E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, the resulting distribution is statistically close to the previous hybrid. Thus, $\mathbf{Hyb}_1 \approx_s \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: In this hybrid, it samples $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ instead of $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$ as in the previous hybrid. By the IND-CPA security of CKKS, $\mathbf{Hyb}_2 \approx_c \mathbf{Hyb}_3$.

Note that $\mathbf{Hyb}_3$ corresponds to the simulator $\mathtt{Sim}^2_{\mathrm{New2PC}}(y, z)$. Combining the above hybrid arguments, Π_{New2PC} securely realizes the functionality $\mathcal{F}_{\mathrm{New2PC}}$. □

4.2 Approximate MPC with Standard Security

We extend the secure approximate 2PC protocol to a multi-party setting by integrating collaborative sampling among multiple parties. In the following, we propose the protocol for an n-out-of-n case. Throughout this subsection, we omit the prefix ThCKKS from the algorithms of the threshold CKKS scheme when it is clear from context. Note that this framework can be generalized to a t-out-of-n threshold setting by incorporating secret-sharing techniques, such as Shamir's secret sharing, to enable threshold decryption. We provide the formal description in Appendix D.

Recent works [51, 53] utilize the threshold CKKS scheme to construct a secure approximate MPC protocol. In the decryption process, all participating parties incorporate smudging noise to their partial decryptions. However, existing works lack rigorous security proof or formal functionality explanation. Since each party integrates an auxiliary smudging noise to the partial decryption in existing protocols, they do not achieve standard MPC security, as mentioned in Section 3.2.

To overcome this limitation, we also apply the collaborative sampling approach, similar to the technique described in Section 4.1, to generate smudging noise jointly. The functionality of the collaborative sampling for n parties, which produces additive shares of a smudging error, is illustrated in Fig. 8.

Functionality $\mathcal{F}_{n\text{-ColSamp}}$

Setup:
Let $\mathcal{D}$ be a smudging error distribution over R.

1. Sample $e \leftarrow \mathcal{D}$.
2. Sample $r_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ for $1 \leq i \leq n-1$, and let $r_n := e - \sum_{i=1}^{n-1} r_i \pmod{q_{\mathsf{out}}}$.
3. Output r_i to party P_i for $1 \leq i \leq n$.

Fig. 8. Functionality of multi-party collaborative sampling.

Through the collaborative sampling technique, each party obtains an additive share r_i of the smudging noise e, which is incorporated into the partial decryption process during the decryption phase. Note that it differs from the existing method in that each party samples its own additional noise from the smudging distribution and adds it to the partial decryption result. The functionality of collaborative sampling ensures that no participating party can obtain any partial information about the ciphertext except the final output of the protocol. Consequently, an approximate MPC protocol incorporating a collaborative smudging error sampling mechanism satisfies the standard MPC security definition. The formal definitions of the functionality and the approximate MPC protocol in this setting are provided in Fig. 9 and Fig. 10, respectively. Now, we show that the approximate MPC protocol Π_{NewMPC} is secure in the $\mathcal{F}_{n\text{-ColSamp}}$-hybrid model.

Functionality $\mathcal{F}_{\text{NewMPC}}$

Setup:
Let C be an evaluation circuit and $\mathcal{D}$ be an error distribution. The functionality receives x_i from n parties P_i $(1 \leq i \leq n)$.

1. Compute $\hat{z} \leftarrow C(x_1, \ldots, x_n)$.
2. Sample $e \leftarrow \mathcal{D}$ and let $z := \hat{z} + e$.
3. Output z to all parties.

Fig. 9. Functionality of the secure approximate MPC.

Protocol Π_{NewMPC}

Offline:
The parties call $\mathcal{F}_{n\text{-ColSamp}}$ and each party P_i receives $r_i \in R_{q_{\mathsf{out}}}$.

. . . . . .

Online:

1. All parties generate $(\mathsf{sk}_i, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. Each party P_i encrypts $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$ and broadcasts it.
3. All parties compute $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$.
   We denote $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}})$.
4. Each party P_i computes $u_i = a_{\mathsf{out}} \cdot s_i + r_i \pmod{q_{\mathsf{out}}}$ and broadcasts it.
5. All parties output $z_{\mathsf{out}} = b_{\mathsf{out}} + \sum_{i=1}^{n} u_i \pmod{q_{\mathsf{out}}}$.

Fig. 10. CKKS-based secure approximate MPC protocol.

Theorem 3. *For plaintexts x_i's and a circuit C, suppose that $\|x_i\| \leq B_i$ for some $B_i > 0$ and let $E := \mathsf{Est}(C, \{B_i\}_{i \leq n})$. If* ThCKKS *is* IND-CPA *secure and $\mathcal{D}$ is an E-smudging distribution, then the MPC protocol Π_{NewMPC} securely realizes the functionality $\mathcal{F}_{NewMPC}$ in the $\mathcal{F}_{n\text{-ColSamp}}$-hybrid model in a standard sense.*

Proof. We sketch a proof and refer to Appendix E. For the correctness, the decryption output can be written as $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$, where $\hat{z} = C(x_1, \ldots, x_n)$ and $\|e_{\mathsf{out}}\| \leq E$. Each party adds its own noise share, so the final result is $z_{\mathsf{out}} = z_{\mathsf{dec}} + e = \hat{z} + e_{\mathsf{out}} + e$ for some $e \leftarrow \mathcal{D}$. Since $\mathcal{D}$ is an E-smudging distribution, z_{out} is statistically indistinguishable from the ideal output $z = \hat{z} + e$.

For security, assume a set A of corrupted parties and H the honest ones. The simulator replaces the honest parties' inputs with encryptions of zero, and for each corrupted party generates partial decryptions $u_i = a_{\mathsf{out}} \cdot s_i + r_i$. Honest parties' shares are sampled uniformly, except the last honest party P_h, whose share is set so that $z_{\mathsf{out}} = b_{\mathsf{out}} + \sum_{i=1}^{n} u_i$ holds. This ensures that the simulated output is consistent with the ideal functionality. The E-smudging property hides the true error, and the IND-CPA security of CKKS makes encryptions of real inputs indistinguishable from encryptions of zero. Hence the adversary's view is indistinguishable from a real execution. □

4.3 Collaborative Sampling Protocols

In this section, we present concrete collaborative sampling protocols that realize the ideal functionality shown in Fig. 5 or Fig. 9. We describe a method corresponding to the uniform distribution which is the most commonly used for smudging distribution. Suppose that each party holds additive shares $r_{i,j}$ of k random bits $b_j \in \{0, 1\}$ for $(0 \leq j \leq k-1)$ where k is determined by the bound of the smudging error. Recall that $\mathcal{D} = [-E \cdot 2^{\lambda-1}, E \cdot 2^{\lambda-1})$ is an E-smudging distribution parameterized by the security parameter λ, where $E = \mathsf{Est}(C, \{B_i\}_{i \leq n})$. We now present a distributed protocol for sampling a smudging error from the uniform distribution using these shared random bits.

In collaborative sampling, we consume $k = \log(E) + \lambda$ additive shares of random bits to sample a smudging error from the distribution $\mathcal{D}$. Specifically, each party P_i locally computes $r_i = \sum_{j=0}^{k-1} 2^j r_{i,j}$ to obtain an additive share of an error e uniformly sampled from $\mathcal{D}$. It can also be naturally extended to collaborative sampling of a smudging error over R, where each coefficient is uniformly sampled from $\mathcal{D}$.

We note that Dahl et al. [21] provided the random-bit sharing protocol assuming a generic MPC protocol. However, this approach inevitably incurs additional communication overhead, which may diminish the practical advantages of CKKS-based protocol, particularly in the multi-party setting or high-latency network environments. To address this issue, we propose round-efficient collaborative sampling algorithms in both two-party and multi-party settings.

Two-party Bit-sharing. In the 2PC case, two parties P_1 and P_2 can generate an additive share of a random bit using a 1-out-of-2 OT protocol as follows:

- P_1 samples $r_1 \xleftarrow{\$} \mathbb{Z}_q$ and a bit $b \xleftarrow{\$} \{0, 1\}$, and sets $(m_0, m_1) = (b - r_1, 1 - b - r_1) \pmod q$
- P_2 samples a selection bit $\sigma \xleftarrow{\$} \{0, 1\}$.
- The parties run OT protocols with inputs (m_0, m_1) from P_1 and σ from P_2, so that P_2 receives m_σ.

The resulting values $(r_1, r_2 := m_\sigma)$ form a random additive share of a random bit, realizing the functionality of collaborative uniform sampling from $\{0, 1\}$.

We instantiate the 1-out-of-2 OT by first executing random OT between P_1 and P_2, and then applying Beaver's OT derandomization technique [6]. The random OT outputs serve as one-time pads, allowing random OT to be converted into sender-chosen OT with a single additional round of communication.

Multi-party Bit-sharing. For the multi-party case, we describe an implementation of collaborative sampling using discrete CKKS [5] in Fig. 11. Note that protocols for obtaining shares of various randomness using discrete CKKS were explored by [49]. We adapt their protocol for our collaborative sampling. Note that while we use CKKS in this protocol, it's security is unaffected from our discussion above as we use a *discrete* version of it. Therefore, we can treat it as an exact HE scheme rather than an approximate one. For more details, we refer to [49, Section 4.1].

Protocol $\Pi_{n\text{-ColSamp}}$

1. All parties generate $(\mathsf{sk}_i, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. Each party P_i samples PRF key $k_{i,j}$ for $j = 0, \ldots, n-1$, and sends $k_{i,j}$ to P_j.
3. Each party P_i samples $b_j \leftarrow \{0,1\}$ for $j = 0, \ldots, N-1$, sets $\mu_i = \sum_{j=0}^{N-1} \Delta_0 b_j X^j$ for $q_0 | Q$ and $\Delta_0 = \lceil \frac{q_0}{2} \rfloor$, and broadcasts $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(\mu_i) \pmod{q_0}$.
4. Upon receiving ct_i, all parties compute $\mathsf{ct}_{\text{rand}} \leftarrow \mathtt{BinBoot}(\sum_{i=1}^{n} \mathsf{ct}_i)$ and $\mathsf{ct}_{\text{out}} \leftarrow \mathtt{Clean}(\mathsf{ct}_{\text{rand}})$. We denote $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}})$.
5. Each party P_i samples $e'_i, e_i \leftarrow \mathcal{D}$, computes $u_i = a_{\mathsf{out}} \cdot s_i + \Delta_0 \mathsf{Ecd}(e'_i) + e_i$ and broadcasts it.
6. All parties compute $m = \lceil \Delta_0^{-1} \mathtt{Dcd}(b_{\mathsf{out}} + \sum_{i=1}^{n} u_i) \rfloor \pmod{q_{\mathsf{out}}}$, and obtain $r_i = -e'_i + n^{-1}m + \sum_{j=1}^{n} \mathtt{PRF}(k_{i,j}, \mathsf{ct}_{\mathsf{out}}) - \mathtt{PRF}(k_{j,i}, \mathsf{ct}_{\mathsf{out}}) \pmod{q_{\mathsf{out}}}$

Fig. 11. A Collaborative Sampling Protocol using Discrete CKKS.

In the protocol, each party P_i samples a polynomial b_i whose coefficients are uniformly random in $\mathbb{Z}_2$, and then broadcasts the encryption of $\mu_i = \lceil q_0/2 \rfloor b_i$ modulo q_0. By simply aggregating them, we obtain an encryption of $b = \sum_i b_i$, whose coefficients are uniformly random over $\mathbb{Z}_2$. However, since the ciphertext modulus is q_0, the plaintext modulus that holds b is 2, whereas we need uniformly random bits over $\mathbb{Z}_{q_{\mathsf{out}}}$ (or $R_{q_{\mathsf{out}}}$). Then, we utilize the discrete CKKS bootstrapping from [4] to enlarge the plaintext modulus to q_{out}. Moreover, we apply cleaning to account for flooding noise during distributed decryption.

After decryption, the parties should obtain additive shares of b that appear uniformly random over $\mathbb{Z}_{q_{\mathsf{out}}}$, rather than b itself. To hide the information about b, each party generates and adds additional flooding noise e', so that all parties obtain the message $m_i = \sum e'_i + b$. Since e'_i is a flooding noise, it is not uniformly random modulo q_{out}, therefore we also use the masking technique [23] to generate a additive share of zero by securely exchanging PRF keys $k_{i,j}$.

As described in [58], secure distributed decryption in the synchronous setting can be achieved by having each party exchange pairwise PRF seeds. Moreover, secure distributed decryption in the asynchronous setting additionally requires a key-homomorphic PRF [26]. Since both the PRF and the corresponding seeds are already necessary components in the synchronous and asynchronous settings, respectively, the proposed masking technique introduces essentially no additional overhead to our protocol.

Benchmark Results for Bit Sharing. We implement and benchmark the protocols for collaborative sampling in both two-party and multi-party cases. All experiments were performed with a single thread on a machine with an Intel(R) Xeon(R) Platinum 8268 CPU running at 2.90GHz and 384GB of RAM.

For the two-party case, we used LibOTe [55] library with the OT extension [12] to generate bit shares for $\log q_{\mathsf{out}} \approx 128$. For the multi-party case, we used Lattigo [1] library to implement discrete CKKS. We use the parameters $N = 2^{16}, \log QP \approx 1200$, so that a single protocol creates 2^{16} bit shares. We also set the parameters so that after the protocol 157 bits of modulus is left with $\log \Delta \approx 60$ and $\approx 2^{-47}$ bits of precision. Hence, we can set $\log q_{\mathsf{out}} \approx 90$ and support up to $2^7 = 128$ parties with 40 bits of statistical security. We also note that we only implement the bootstrapping and the cleaning part, as it is the main bottleneck of the protocol.

#(bit-shares)	Setting	time (s)
2^{16}	2PC	0.123
	MPC	33.96
2^{18}	2PC	0.408
	MPC	135.84

Table 1. Benchmark results for bit-sharing

5 Approximate Computation Protocols with Liberal Security

Under the standard security model, the simulator for approximate MPC is required to reproduce the adversary's view using only the adversary's input and the ideal functionality output, which is an approximation of the computation result. However, we demonstrate that existing CKKS-based 2PC and MPC protocols do not achieve standard simulation-based security in Sec. 3. Nevertheless, we observe that these protocols satisfy a weaker security notion known as *liberal security*, inspired by Feigenbaum et al. [27]. The liberal definition allows the simulator to obtain additional auxiliary information, such as the exact value of the target function in an approximate computation protocol. Therefore, the protocol provides liberal security guarantees that it does not reveal more information than derived from the ideal functionality and auxiliary information. We define the notion of liberal security and discuss the implications, while also clarifying its differences from standard security and the liberal security notion introduced in [27].

Under our definition, we also show that the existing 2PC and MPC protocols (Fig. 1 and Fig. 3) satisfy liberal security, despite failing to achieve standard simulation-based security. We note that this security notion introduces a trade-off, particularly between the efficiency and precision of the protocol, as will be discussed later. From this perspective, the construction of liberally secure protocols remains meaningful in certain scenarios.

5.1 Definition of Liberal Security

Standard security for MPC requires that the joint distribution of the simulator's output and the ideal functionality be indistinguishable from the joint distribution of the corrupted party's view and the parties' outputs in the real execution. Meanwhile, we introduce a liberal security definition for secure approximations by extending the standard definition of (exact) MPC security. Intuitively, under the liberal security model, the simulator is provided not only the ideal functionality f but also auxiliary information $\hat{f}$. A protocol is said to achieve liberal security if there exists a simulator that can generate the valid adversary's view using both f and $\hat{f}$. We formalize this notion of liberal security as below.

Definition 4 (Secure Multi-party Approximation: Liberal Definition). *Let Π be an n-party protocol for computing a functionality $f = (f_1, \ldots, f_n)$. For a set of indices $A \subseteq [n]$, we denote $f_A(\boldsymbol{x}) = (f_i(\boldsymbol{x}))_{i\in A}$, $\mathsf{view}_A^{\Pi}(\boldsymbol{x}) = (\mathsf{view}_i^{\Pi}(\boldsymbol{x}))_{i\in A}$, $\hat{f}(\boldsymbol{x}) = (\hat{f}_j(\boldsymbol{x}))_{j\in[n]}$, and $\mathsf{output}^{\Pi}(\boldsymbol{x}) = (\mathsf{output}_j^{\Pi}(\boldsymbol{x}))_{j\in[n]}$ with a valid input $\boldsymbol{x} = (x_1, \ldots, x_n)$. We also define an auxiliary information $\hat{f} = (\hat{f}_1, \ldots, \hat{f}_n)$ for the functionality f. We say that a protocol Π securely computes f in the presence of semi-honest adversaries* in the liberal sense*, if there exist PPT algorithms $\mathtt{Sim}^{\Pi}$ such that for every $A \subseteq [n]$ and $\boldsymbol{x}$, the following holds.*

$$\{(\mathtt{Sim}^{\Pi}(A, (x_i)_{i\in A}, f_A(\boldsymbol{x}), \hat{f}_A(\boldsymbol{x})), f(\boldsymbol{x}))\} \approx_c \{(\mathsf{view}_A^{\Pi}(\boldsymbol{x}), \mathsf{output}^{\Pi}(\boldsymbol{x}))\}.$$

Our definition of liberal security guarantees that the adversary cannot obtain more information than the adversary that is given access to both the ideal functionality f and the auxiliary information $\hat{f}$. It constitutes the primary distinction from the standard MPC security definition. More precisely, liberal security guarantees that the protocol reveals no additional private information beyond that already disclosed by the pair $(f, \hat{f})$, whereas the standard simulation-based definition restricts leakage solely to the information revealed by the ideal functionality f.

Previous work [27] also introduced a notion of liberal security for approximate MPC by formalizing auxiliary information through the concept of a *functionally private approximation* of a target deterministic function. Informally, f is said to be a functionally private approximation of $\hat{f}$ if there exists a PPT sampling algorithm $\mathcal{S}$ such that the distribution $\mathcal{S}(\hat{f}(\boldsymbol{x}))$ is indistinguishable from $f(\boldsymbol{x})$ for every input $\boldsymbol{x}$. Under this assumption, a corrupted party possessing only $\hat{f}$ has essentially the same adversarial capability as a party that knows both ideal functionality f and auxiliary information $\hat{f}$. Therefore, the liberal security definition, together with the condition that f is a functionally private approximation of $\hat{f}$, guarantees that the approximate computation protocol does not reveal more information beyond what is already disclosed by only the auxiliary information itself. It is the main difference between our liberal security and the original definition. Our definition guarantees that the protocol leaks no more information than

can be derived from $(f, \hat{f})$. On the other hand, the previous definition ensures that the protocol leakage of the protocol does not exceed the information revealed by $\hat{f}$ alone, by assuming from the outset that f is a functionally private approximation of $\hat{f}$.

5.2 Approximate 2PC with Liberal Security

We mention that $\mathsf{IND\text{-}CPA^D}$ and circuit privacy provide only one-sided security rather than ensuring the security of the entire protocol. To address the security vulnerabilities, we consider a protocol that achieves both security using noise smudging. In Sec. 3, we presented the ideal functionality $\mathcal{F}_{\mathrm{Prev2PC}}$ for approximate 2PC (Fig. 2) and a concrete protocol Π_{Prev2PC} realizing it (Fig. 1). In this subsection, we prove the correctness and liberal security of Π_{Prev2PC}. For the liberal security analysis, we designate the auxiliary information $\hat{f}$ as a functionality that outputs an exact computation result $\hat{z}$ of the circuit. Throughout this subsection, we omit the prefix `CKKS` from the algorithms of the threshold CKKS scheme when it is clear from context.

Theorem 4. *For plaintexts x, y and a circuit C, suppose that $\|x\| \leq B_1$ and $\|y\| \leq B_2$ for some $B_1, B_2 > 0$. Let $B_0 > 0$ be an upper bound of public-key encryption error, i.e., $\Pr[\|\mathtt{Dec}_{\mathsf{sk}}(\mathtt{Enc}_{\mathsf{pk}}(0))\| \leq B_0]$ with an overwhelming probability. Let $E := \mathsf{Est}(C, B_1, B_2) + B_0$. If `CKKS` is $\mathsf{IND\text{-}CPA}$ secure and $\mathcal{D}$ is an E-smudging distribution, then the 2PC protocol $\Pi_{Prev2PC}$ securely realizes the functionality $\mathcal{F}_{Prev2PC}$ in the liberal sense.*

Proof. We first show the correctness of Π_{Prev2PC}. Let $\hat{z} := C(x, y)$ and denote $z_{\mathsf{dec}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}})$ in the protocol. From the error estimator Est, we have $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ for some $e_{\mathsf{out}} \in R$ such that $\|e_{\mathsf{out}}\| \leq \mathsf{Est}(C, B_1, B_2) + B_0 = E$. Then, the output z_{out} can be written as $z_{\mathsf{out}} = \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}}) + e_1 + e_2 = \hat{z} + e_{\mathsf{out}} + e_1 + e_2$ for some $e_1, e_2 \leftarrow \mathcal{D}$. Thus, the distribution of this output z_{out} is statistically indistinguishable from the distribution of the output $z = \hat{z} + e_1 + e_2$ of the functionality $\mathcal{F}_{\mathrm{Prev2PC}}$, since $\mathcal{D}$ is an E-smudging distribution.

To prove the liberal security of Π_{Prev2PC}, we separately consider a corrupted client P_1 and a corrupted server P_2, using a sequence of hybrid arguments for each corruption case. For convenience, we use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted party's view and the protocol output.

We first consider a corrupted client P_1. The simulator $\mathtt{Sim}^1_{\mathrm{Prev2PC}}(x, \hat{z}, z)$ for P_1's view proceeds as follows:

1. Generate $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$.
2. Let $e := z - \hat{z}$ and sample $(e_1, e_2) \leftarrow \{(e_1, e_2) \leftarrow \mathcal{D} \times \mathcal{D} \mid e_1 + e_2 = e\}$.
3. Sample $a \xleftarrow{\$} R_{q_{\mathsf{out}}}$ and let $\mathsf{ct}'_{\mathsf{out}} := (\hat{z} + e_2 - as, a)$.
4. Output $(x, \mathsf{sk}, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}'_{\mathsf{out}}, e_1)$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world scenario. The joint distribution of P_1's view and the protocol output is as follows:

$$
\begin{aligned}
&\{(\mathsf{view}_1^{\Pi_{\mathrm{Prev2PC}}}(x, y), \mathsf{output}_2^{\Pi_{\mathrm{Prev2PC}}}(x, y))\} \\
\equiv&\{(x, \mathsf{sk}, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}'_{\mathsf{out}} = \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (e_2, 0), e_1, z_{\mathsf{dec}} + e_1 + e_2) \mid \\
&(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x), \quad e_1, e_2 \leftarrow \mathcal{D}, \\
&\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}, y), \quad \mathsf{ct}_{\mathsf{zero}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0), \quad z_{\mathsf{dec}} = \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}})\}
\end{aligned}
$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that every occurrence of e_2 in the received ciphertext and the protocol output is replaced by $e_2 - e_{\mathsf{out}}$. Namely, it sets $\mathsf{ct}'_{\mathsf{out}} = \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (e_2 - e_{\mathsf{out}}, 0)$ and $z_{\mathsf{dec}} + e_1 + e_2 - e_{\mathsf{out}}$. By approximate correctness, $\|e_{\mathsf{out}}\| \leq E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is an algebraic rewriting of $\mathbf{Hyb}_1$. Since $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$, the protocol output is $z_{\mathsf{dec}} + e_1 + (e_2 - e_{\mathsf{out}}) = \hat{z} + e_1 + e_2$. Moreover, writing $\mathsf{ct}_{\mathsf{out}} = (b_{\mathsf{out}}, a_{\mathsf{out}})$ and $\mathsf{ct}_{\mathsf{zero}} = (b_{\mathsf{zero}}, a_{\mathsf{zero}})$, we have

$b_{\mathsf{out}} + b_{\mathsf{zero}} = \hat{z} + e_{\mathsf{out}} - (a_{\mathsf{out}} + a_{\mathsf{zero}})s$. Therefore, $\mathsf{ct}'_{\mathsf{out}} = \mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}} + (e_2 - e_{\mathsf{out}}, 0) = (\hat{z} + e_2 - (a_{\mathsf{out}} + a_{\mathsf{zero}})s, a_{\mathsf{out}} + a_{\mathsf{zero}})$. Thus, no distributional change occurs, and $\mathbf{Hyb}_1 \equiv \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid is identical to $\mathbf{Hyb}_2$, except that a_{zero} of the fresh zero encryption $\mathsf{ct}_{\mathsf{zero}} = (b_{\mathsf{zero}}, a_{\mathsf{zero}}) \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ is replaced by an independently sampled $\widetilde{a}_{\mathsf{zero}} \xleftarrow{\$} R_{q_{\mathsf{out}}}$. Since a_{zero} is an RLWE sample with fresh short secret and error, it is computationally indistinguishable from uniform under the decisional RLWE assumption. The received ciphertext is accordingly generated as $\mathsf{ct}'_{\mathsf{out}} := \big(\hat{z} + e_2 - (a_{\mathsf{out}} + \widetilde{a}_{\mathsf{zero}})s,\ a_{\mathsf{out}} + \widetilde{a}_{\mathsf{zero}}\big)$. Hence, $\mathbf{Hyb}_2 \approx_c \mathbf{Hyb}_3$.

$\mathbf{Hyb}_4$: Since $\widetilde{a}_{\mathsf{zero}}$ is uniform over $R_{q_{\mathsf{out}}}$, the value $a := a_{\mathsf{out}} + \widetilde{a}_{\mathsf{zero}}$ is also uniform over $R_{q_{\mathsf{out}}}$, even conditioned on a_{out}. Therefore, we rewrite the received ciphertext as $\mathsf{ct}'_{\mathsf{out}} = \big(\hat{z} + e_2 - as, a\big)$, where $a \xleftarrow{\$} R_{q_{\mathsf{out}}}$. Thus, $\mathbf{Hyb}_3 \equiv \mathbf{Hyb}_4$.

$\mathbf{Hyb}_5$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{Prev2PC}}(x, y)$ and $\hat{z} = \hat{f}(x, y)$, computes $e := z - \hat{z}$ and samples $(e_1, e_2) \leftarrow \{(e_1, e_2) \leftarrow \mathcal{D} \times \mathcal{D} \mid e_1 + e_2 = e\}$. By the definition of $\mathcal{F}_{\mathrm{Prev2PC}}$, the ideal output is generated as $z = \hat{z} + e_1 + e_2$ for independently sampled $e_1, e_2 \leftarrow \mathcal{D}$. Thus, sampling the pair conditioned on $e_1 + e_2 = z - \hat{z}$ is merely an equivalent reparameterization of the distribution in $\mathbf{Hyb}_4$. The resulting distribution is identical to $\mathbf{Hyb}_4$. Thus, $\mathbf{Hyb}_4 \equiv \mathbf{Hyb}_5$. Note that $\mathbf{Hyb}_5$ corresponds to the simulator $\mathtt{Sim}^1_{\mathrm{Prev2PC}}(x, \hat{z}, z)$.

Next, we construct $\mathtt{Sim}^2_{\mathrm{Prev2PC}}(y, \hat{z}, z)$ that simulates P_2's view in the protocol as follows:

1. Generate $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
2. Compute $\mathsf{ct}_{\mathsf{zero}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
3. Compute $e := z - \hat{z}$ and sample $(e_1, e_2) \leftarrow \{(e_1, e_2) \leftarrow \mathcal{D} \times \mathcal{D} \mid e_1 + e_2 = e\}$.
4. Output $(y, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}_{\mathsf{zero}}, e_2)$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world scenario. The joint distribution of P_2's view and the protocol output is as follows:

$$
\begin{aligned}
&\{(\mathsf{view}_2^{\Pi_{\mathrm{Prev2PC}}}(x, y), \mathsf{output}_1^{\Pi_{\mathrm{Prev2PC}}}(x, y))\} \\
\equiv&\{(y, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}_{\mathsf{zero}}, e_2, z_{\mathsf{out}}) \mid (\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), e_1, e_2 \leftarrow \mathcal{D}, \\
&\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x), \mathsf{ct}_{\mathsf{zero}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0), \mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}, y), \\
&z_{\mathsf{out}} = \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}} + \mathsf{ct}_{\mathsf{zero}}) + e_1 + e_2\}
\end{aligned}
$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that the hidden error e_1 contributing to the protocol output is replaced by $e_1 - e_{\mathsf{out}}$. Accordingly, the output becomes $z_{\mathsf{dec}} + (e_1 - e_{\mathsf{out}}) + e_2 = \hat{z} + e_1 + e_2$. By approximate correctness, $\|e_{\mathsf{out}}\| \le E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, we have $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is identical to $\mathbf{Hyb}_1$, except that it replaces $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$ with $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$. By the IND-CPA security of CKKS, $\mathbf{Hyb}_1 \approx_c \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{Prev2PC}}(x, y)$ and $\hat{z} = \hat{f}(x, y)$, computes $e := z - \hat{z}$ and samples $(e_1, e_2) \leftarrow \{(e_1, e_2) \leftarrow \mathcal{D} \times \mathcal{D} \mid e_1 + e_2 = e\}$. By the definition of $\mathcal{F}_{\mathrm{Prev2PC}}$, the ideal output is generated as $z = \hat{z} + e_1 + e_2$ for independently sampled $e_1, e_2 \leftarrow \mathcal{D}$. Thus, sampling the pair conditioned on $e_1 + e_2 = z - \hat{z}$ is merely an equivalent reparameterization of the distribution in $\mathbf{Hyb}_2$. Thus, $\mathbf{Hyb}_2 \equiv \mathbf{Hyb}_3$. We also note that $\mathbf{Hyb}_3$ corresponds to the simulator $\mathtt{Sim}^2_{\mathrm{Prev2PC}}(y, \hat{z}, z)$. Combining the above hybrid arguments, Π_{Prev2PC} securely realizes the functionality $\mathcal{F}_{\mathrm{Prev2PC}}$ in the liberal sense. □

Refer to [27], the ideal functionality f is a functionally private approximation of $\hat{f}$ that outputs the exact result $\hat{z}$. As a result, we can also ensure that the adversary is unable to infer more information from the existing 2PC protocol than can be obtained from the exact computation result $\hat{z}$.

5.3 Approximate MPC with Liberal Security

In the CKKS-based MPC protocol, all participating parties collaboratively generate a shared public key and construct additive shares of the secret key during the key generation phase. When parties decrypt

the result ciphertext, each party computes a partial decryption and adds an extra noise, which is sampled from the smudging distribution. We present an ideal functionality $\mathcal{F}_{\text{PrevMPC}}$ (Fig. 4) of this scenario and provide a concrete protocol Π_{PrevMPC} to realize the functionality.

We also discussed that the protocol Π_{PrevMPC} is insecure under the standard MPC security definition, as it is infeasible to validate simulators without access to the exact computation result in the previous section (Sec. 3). In the following, we provide a formal security proof that the existing MPC protocols adopting the threshold CKKS scheme satisfy a liberal definition of a secure MPC protocol. Throughout the proof, we also assume that $\hat{f}$ outputs an exact computation result $\hat{z}$ of the circuit and we omit the prefix $\texttt{ThCKKS}$ from the algorithms of the CKKS scheme when it is clear from context.

Theorem 5. *For plaintexts x_i's and a circuit C, suppose that $\|x_i\| \leq B_i$ for some $B_i > 0$ and let $E := \mathsf{Est}(C, \{B_i\}_{i \leq n})$. If $\texttt{ThCKKS}$ is $\mathsf{IND\text{-}CPA}$ secure and $\mathcal{D}$ is an E-smudging distribution, then the MPC protocol $\Pi_{PrevMPC}$ securely realizes the functionality $\mathcal{F}_{PrevMPC}$ in the liberal sense.*

Proof. We sketch a proof of Theorem 5 and refer to Appendix F. As in the two-party case, we introduce the smudging error e_i to conceal the decryption error that each party P_i adds during the protocol.

We first show the correctness of the protocol Π_{PrevMPC}. We denote $\hat{z} = C(x_1, \ldots, x_n)$ and $z_{\mathsf{dec}} = \mathsf{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}}) := b_{\mathsf{out}} + \sum_{i=1}^{n} a_{\mathsf{out}} \cdot s_i$. By the definition of the error estimator Est, we have $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ for some $e_{\mathsf{out}} \in R$ such that $\|e_{\mathsf{out}}\| \leq E$. By the linearity of the decryption process, the output can be represented as follows: $z_{\mathsf{out}} = b_{\mathsf{out}} + \sum_{i=1}^{n} u_i = b_{\mathsf{out}} + \sum_{i=1}^{n} (a_{\mathsf{out}} \cdot s_i + e_i) \pmod{q_{\mathsf{out}}} = z_{\mathsf{dec}} + e = \hat{z} + e_{\mathsf{out}} + e$ for some $e \leftarrow \mathcal{D}$. Since $\mathcal{D}$ is an E-smudging distribution, the distribution of the protocol's output z_{out} is statistically indistinguishable from the output distribution $z = \hat{z} + e$ in the ideal functionality.

To prove the security of Π_{PrevMPC}, we assume the case where the adversary corrupts exactly $n - 1$ parties. We denote a party P_h as the only honest party and $A := [n] \backslash \{h\}$ as the set of adversarial parties' indices. We construct a simulator for the adversarial parties. Since A knows $(x_i)_{i \in A}$, it encrypts $(x_i)_{i \in A}$ for adversaries and zero for the honest party. To emulate $\mathcal{F}_{\text{PrevMPC}}$, the simulator samples r_i uniformly from $R_{q_{\mathsf{out}}}$ and compute partial decryption $u_i := a_{\mathsf{out}} \cdot s_i + r_i$ for $i \in A$. For the honest party, the simulator computes $u_h := z_{\mathsf{out}} - b_{\mathsf{out}} - \sum_{i \in A} u_i$, so that $z_{\mathsf{out}} = b_{\mathsf{out}} + \sum_{i=1}^{n} u_i$ holds. Now, by the E-smudging property and the $\mathsf{IND\text{-}CPA}$ security of CKKS, the simulated view is indistinguishable from the real protocol. □

Similar to the 2PC case, we can guarantee that the existing MPC protocol does not reveal additional information beyond what is implied by the exact computation result, since f is a functionally private approximation of $\hat{f}$.

Efficiency Comparison between Liberal and Standard Protocol. The design differences between the liberal and standard protocols lead to differences in both efficiency and precision. In the standard protocol, all parties participate in a collaborative sampling protocol that generates a shared smudging error. As a result, constructing an approximate MPC protocol with standard security incurs additional computational overhead compared with the approximate MPC that achieves liberal security.

In contrast, in the liberal protocol, each party independently samples and adds smudging noise to its own partial decryption. Therefore, collaborative sampling in the standard protocol reduces the total noise by a factor of the number of parties, improving precision and reducing the error size at the expense of additional overhead from the collaborative sampling procedure.

Discussion about Leakage-Tolerance. Several studies have investigated the security of protocols in the presence of additional information, commonly referred to as leakage [8, 29, 35]. In contrast to the classical security model, where an adversary is assumed to observe only the inputs and outputs of an algorithm, practical implementations inevitably expose side-channel information in real-world environments. These works considered leakage-tolerant protocols, which guarantee that the adversary gains no greater advantage from the leaked information than it would obtain by observing only the protocol inputs and outputs in the absence of the leakage.

Note that the liberal security guarantees only that the protocol does not reveal information beyond the ideal functionality and the auxiliary information provided to the simulator. However, this notion alone does not imply that the pair of $(f, \hat{f})$ discloses no more leakage than $\hat{f}$ itself. Therefore, to argue that the protocol reveals no more information than the exact computation result, it is necessary to ensure that the

ideal functionality does not reveal extra information compared to the auxiliary information. To address this requirement, [27] assumes that the ideal functionality f is functionally private approximation of $\hat{f}$.

The concept of leakage-tolerance can be applied to establish the security of liberally secure protocols by interpreting the auxiliary information as leakage. In particular, if the liberally secure protocol additionally satisfies the leakage-tolerance, then the adversary cannot obtain additional information beyond the ideal functionality f. However, previous works [8,29,35] primarily considered leakage as additional information that the adversary actually can obtain, whereas the exact computation result (regarded as auxiliary information in our proof) cannot generally be obtained through such attacks. Bridging this conceptual gap remains an interesting direction for future research.

6 Conclusion

In this work, we show that existing CKKS-based 2PC and MPC protocols fail to achieve the standard simulation-based notion of MPC security. To address this limitation, we introduce a collaborative sampling technique for noise smudging and construct CKKS-based protocols that achieve the standard security definition by integrating this technique. In addition, we generalize the notion of liberal security as a relaxed security definition relative to the standard MPC notion, and demonstrate that existing CKKS-based protocols for approximate computation satisfy this liberal security definition.

Acknowledgement

This work was supported by Samsung Research Funding & Incubation Center of Samsung Electronics under Project Number SRFC-TB2103-01. Y. Song was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2026-25619347). M. Kim was supported by the NRF grants funded by the Korean government (MSIT) (RS-2026-25495438, RS-2026-25518141); Basic Science Research Program through the NRF funded by the Ministry of Education (2020R1A6A1A06046728); and Korea Basic Science Institute (National Research Facilities and Equipment Center) grant funded by the Ministry of Education (2023R1A6C101A009). Y. Hwang and D. Lee were supported by Basic Science Research Program through the NRF funded by the Ministry of Education (RS-2025-25435180, RS-2025-25435347). This paper was edited for grammar and readability using AI tools, ChatGPT and Gemini.

References

1. Lattigo v6. Online: `https://github.com/tuneinsight/lattigo` (Aug 2024), ePFL-LDS, Tune Insight SA
2. Alexandru, A., Al Badawi, A., Micciancio, D., Polyakov, Y.: Application-aware approximate homomorphic encryption. IACR Communications in Cryptology **2**(4) (2026)
3. Asharov, G., Jain, A., López-Alt, A., Tromer, E., Vaikuntanathan, V., Wichs, D.: Multiparty computation with low communication, computation and interaction via threshold fhe. In: Advances in Cryptology–EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings 31. pp. 483–501. Springer, Berlin, Heidelberg (2012)
4. Bae, Y., Cheon, J.H., Kim, J., Stehlé, D.: Bootstrapping bits with ckks. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 94–123. Springer (2024)
5. Bae, Y., Kim, J., Stehlé, D., Suvanto, E.: Bootstrapping small integers with ckks. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 330–360. Springer (2024)
6. Beaver, D.: Precomputing oblivious transfer. In: Annual International Cryptology Conference. pp. 97–109. Springer (1995)
7. Bergamaschi, F., Costache, A., Dachman-Soled, D., Kippen, H., LaBuff, L., Tang, R.: Revisiting the security of approximate FHE with noise-flooding countermeasures. In: IACR International Conference on Public-Key Cryptography. pp. 102–134. Springer, Berlin, Heidelberg (2025)
8. Bitansky, N., Canetti, R., Halevi, S.: Leakage-tolerant interactive protocols. In: Theory of Cryptography Conference. pp. 266–284. Springer (2012)
9. Boneh, D., Gennaro, R., Goldfeder, S., Jain, A., Kim, S., Rasmussen, P.M., Sahai, A.: Threshold cryptosystems from threshold fully homomorphic encryption. In: Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I 38. pp. 565–596. Springer, Berlin, Heidelberg (2018)
10. Bossuat, J.P., Costache, A., Mouchet, C., Nürnberger, L., Troncoso-Pastoriza, J.R.: Accurate and composable noise estimates for CKKS with application to exact HE computation. IACR Communications in Cryptology **2**(2) (2025)
11. Bourse, F., Del Pino, R., Minelli, M., Wee, H.: FHE circuit privacy almost for free. In: Annual International Cryptology Conference. pp. 62–89. Springer, Berlin, Heidelberg (2016)
12. Boyle, E., Couteau, G., Gilboa, N., Ishai, Y., Kohl, L., Rindal, P., Scholl, P.: Efficient two-round ot extension and silent non-interactive secure computation. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 291–308. ACM, New York, NY, USA (2019)
13. Brakerski, Z.: Fully homomorphic encryption without modulus switching from classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) Advances in Cryptology – CRYPTO 2012. pp. 868–886. Springer Berlin Heidelberg, Berlin, Heidelberg (2012)
14. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (leveled) fully homomorphic encryption without bootstrapping. ACM Transactions on Computation Theory (TOCT) **6**(3), 1–36 (2014)
15. Brutzkus, A., Gilad-Bachrach, R., Elisha, O.: Low latency privacy preserving inference. In: International Conference on Machine Learning. pp. 812–821. PMLR (2019)
16. Chen, H., Dai, W., Kim, M., Song, Y.: Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 395–412. ACM, New York, NY, USA (2019)
17. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks against the IND-CPAD security of exact fhe schemes. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. pp. 2505–2519. ACM, New York, NY, USA (2024)
18. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic Encryption for Arithmetic of Approximate Numbers. In: Proc. of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT 2017). pp. 409–437. Springer, Berlin, Heidelberg (2017)
19. Chillotti, I., Gama, N., Georgieva, M., Izabachene, M.: Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In: international conference on the theory and application of cryptology and information security. pp. 3–33. Springer, Berlin, Heidelberg (2016)
20. Costache, A., Curtis, B.R., Hales, E., Murphy, S., Ogilvie, T., Player, R.: On the precision loss in approximate homomorphic encryption. In: International Conference on Selected Areas in Cryptography. pp. 325–345. Springer, Berlin, Heidelberg (2023)
21. Dahl, M., Demmler, D., El Kazdadi, S., Meyre, A., Orfila, J.B., Rotaru, D., Smart, N.P., Tap, S., Walter, M.: Noah's ark: Efficient threshold-fhe using noise flooding. In: Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. pp. 35–46 (2023)

22. Damgård, I., Pastro, V., Smart, N., Zakarias, S.: Multiparty computation from somewhat homomorphic encryption. In: Safavi-Naini, R., Canetti, R. (eds.) Annual Cryptology Conference. pp. 643–662. Springer, Berlin, Heidelberg (2012)
23. Del Pino, R., Katsumata, S., Maller, M., Mouhartem, F., Prest, T., Saarinen, M.J.: Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 219–248. Springer (2024)
24. Ducas, L., Micciancio, D.: Fhew: bootstrapping homomorphic encryption in less than a second. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 617–640. Springer, Berlin, Heidelberg (2015)
25. Ducas, L., Stehlé, D.: Sanitization of fhe ciphertexts. In: Advances in Cryptology–EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8-12, 2016, Proceedings, Part I 35. pp. 294–310. Springer, Berlin, Heidelberg (2016)
26. Ebrahimi, E., Yadav, A.: Strongly secure universal thresholdizer. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 207–239. Springer (2024)
27. Feigenbaum, J., Ishai, Y., Malkin, T., Nissim, K., Strauss, M.J., Wright, R.N.: Secure multiparty computation of approximations. ACM transactions on Algorithms (TALG) **2**(3), 435–472 (2006)
28. Froelicher, D., Cho, H., Edupalli, M., Sousa, J.S., Bossuat, J.P., Pyrgelis, A., Troncoso-Pastoriza, J.R., Berger, B., Hubaux, J.P.: Scalable and privacy-preserving federated principal component analysis. In: 2023 IEEE Symposium on Security and Privacy (SP). pp. 1908–1925. IEEE, New York, NY, USA (2023)
29. Garg, S., Jain, A., Sahai, A.: Leakage-resilient zero knowledge. In: Annual Cryptology Conference. pp. 297–315. Springer (2011)
30. Gentry, C.: Fully Homomorphic Encryption Using Ideal Lattices. In: Proc. of the Forty-First Annual ACM Symposium on Theory of Computing (STOC 2009) (2009). https://doi.org/10.1145/1536414.1536440
31. Gilad-Bachrach, R., Dowlin, N., Laine, K., Lauter, K., Naehrig, M., Wernsing, J.: Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In: International conference on machine learning. pp. 201–210. PMLR (2016)
32. Goldreich, O.: Foundations of Cryptography, Volume 2. Cambridge university press Cambridge (2004)
33. Guo, Q., Nabokov, D., Suvanto, E., Johansson, T.: Key recovery attacks on approximate homomorphic encryption with non-worst-case noise flooding countermeasures. In: Usenix Security (2024)
34. Hwang, I., Min, S., Seo, J., Song, Y.: Practical TFHE ciphertext sanitization for oblivious circuit evaluation. In: Huang, C.Y., Chen, J.C., Shieh, S.P., Lie, D., Cortier, V. (eds.) ACM CCS 2025: 32nd Conference on Computer and Communications Security. pp. 4064–4078. ACM Press, Taipei, Taiwan (October 13–17, 2025). https://doi.org/10.1145/3719027.3765231
35. Ishai, Y., Song, Y.: Leakage-tolerant circuits. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 196–225. Springer (2024)
36. Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., et al.: Advances and open problems in federated learning. Foundations and Trends® in Machine Learning **14**(1–2), 1–210 (2021)
37. Keller, M.: Mp-spdz: A versatile framework for multi-party computation. In: Proceedings of the 2020 ACM SIGSAC conference on computer and communications security. pp. 1575–1590 (2020)
38. Keller, M., Pastro, V., Rotaru, D.: Overdrive: Making spdz great again. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 158–189. Springer, Berlin, Heidelberg (2018)
39. Kim, M., Jiang, X., Lauter, K., Ismayilzada, E., Shams, S.: Secure human action recognition by encrypted neural network inference. Nature communications **13**(1), 4799 (2022)
40. Kim, T., Kwak, H., Lee, D., Seo, J., Song, Y.: Asymptotically faster multi-key homomorphic encryption from homomorphic gadget decomposition. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 726–740. ACM, New York, NY, USA (2023)
41. Kluczniak, K., Santato, G.: On circuit private, multikey and threshold approximate homomorphic encryption. Cryptology ePrint Archive (2023)
42. Kwak, H., Lee, D., Song, Y., Wagh, S.: A general framework of homomorphic encryption for multiple parties with non-interactive key-aggregation. In: International Conference on Applied Cryptography and Network Security. pp. 403–430. Springer, Berlin, Heidelberg (2024)
43. Lee, J.W., Kang, H., Lee, Y., Choi, W., Eom, J., Deryabin, M., Lee, E., Lee, J., Yoo, D., Kim, Y.S., et al.: Privacy-preserving machine learning with fully homomorphic encryption for deep neural network. iEEE Access **10**, 30039–30054 (2022)
44. Li, B., Micciancio, D.: On the security of homomorphic encryption on approximate numbers. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 648–677. Springer, Berlin, Heidelberg (2021)

45. Li, B., Micciancio, D., Schultz-Wu, M., Sorrell, J.: Securing approximate homomorphic encryption using differential privacy. In: Annual International Cryptology Conference. pp. 560–589. Springer, Berlin, Heidelberg (2022)
46. Liu, F., Zheng, Z., Shi, Y., Tong, Y., Zhang, Y.: A survey on federated learning: a perspective from multi-party computation. Frontiers of Computer Science **18**(1), 181336 (2024)
47. Liu, Z., Wang, Y., Fisch, B.: Ind-cpa-d of relaxed functional bootstrapping: A new attack, a general fix, and a stronger model. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 2907–2921 (2025)
48. López-Alt, A., Tromer, E., Vaikuntanathan, V.: On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In: Proceedings of the forty-fourth annual ACM symposium on Theory of computing. pp. 1219–1234. ACM, New York, NY, USA (2012)
49. Min, S., Hanrot, G., Park, J.H., Passelègue, A., Stehlé, D.: Distributed key generation for efficient threshold-CKKS. In: ACM CCS 2026: 33rd Conference on Computer and Communications Security. ACM Press, Hague, Netherlands (November 15–19, 2026). https://doi.org/10.1145/3830454.3846715
50. Moon, J., Yoo, D., Jiang, X., Kim, M.: Thor: Secure transformer inference with homomorphic encryption. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. pp. 3765–3779. ACM, New York, NY, USA (2025)
51. Mouchet, C., Bertrand, E., Hubaux, J.P.: An efficient threshold access-structure for rlwe-based multiparty homomorphic encryption. Journal of Cryptology **36**(2), 10 (2023)
52. Mouchet, C., Chatel, S., Pyrgelis, A., Troncoso, C.: Helium: Scalable MPC among lightweight participants and under churn. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 3038–3052. CCS '24, ACM, New York, NY, USA (2024)
53. Mouchet, C., Troncoso-Pastoriza, J., Bossuat, J.P., Hubaux, J.P.: Multiparty homomorphic encryption from ring-learning-with-errors. Proceedings on Privacy Enhancing Technologies **2021**(4), 291–311 (2021)
54. Mukherjee, P., Wichs, D.: Two round multiparty computation via multi-key fhe. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 735–763. Springer, Berlin, Heidelberg (2016)
55. Peter Rindal, L.R.: libOTe: an efficient, portable, and easy to use Oblivious Transfer Library. https://github.com/osu-crypto/libOTe
56. Qiu, F., Yang, H., Zhou, L., Ma, C., Fang, L.: Privacy preserving federated learning using ckks homomorphic encryption. In: International conference on wireless algorithms, systems, and applications. pp. 427–440. Springer, Berlin, Heidelberg (2022)
57. Smart, N.P.: Practical and efficient fhe-based mpc. In: IMA International Conference on Cryptography and Coding. pp. 263–283. Springer, Berlin, Heidelberg (2023)
58. de Verdière, F.C., Passelègue, A., Stehlé, D.: On threshold fully homomorphic encryption with synchronized decryptors. Cryptology ePrint Archive (2026)

A Definition of IND-CPA^{D} Security

A new security model called IND-CPA^{D} security provides the adversary $\mathcal{A}$ with access to a decryption oracle that decrypts ciphertexts derived from encryption and evaluation of messages of $\mathcal{A}$'s choice. This is formally defined by granting the adversary access to three types of oracles: encryption (E^b_{pk}), evaluation (H^b_{evk}), and decryption (D^b_{pk}), as described in Alg. 1.

Algorithm 1 Oracles for the IND-CPA^{D} game.

Initialize
  $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$
global state
  $S \leftarrow \emptyset$
  $i \leftarrow 0$
Encryption oracle $E^b_{\mathsf{pk}}(m_0, m_1) :=$
  $\mathsf{ct} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(m_b)$
  $S[i] \leftarrow (m_0, m_1, \mathsf{ct})$
  $i \leftarrow i + 1$
  return ct, i
Evaluation oracle $H^b_{\mathsf{evk}}(g, \boldsymbol{J} = (j_i, \ldots, j_k), S) :=$
  $\mathsf{ct} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(g, S[j_1].\mathsf{ct}, \ldots, S[j_k].\mathsf{ct})$
  $g_0 \leftarrow g(S[j_1].m_0, \ldots, S[j_k].m_0)$
  $g_1 \leftarrow g(S[j_1].m_1, \ldots, S[j_k].m_1)$
  $S[i] \leftarrow (g_0, g_1, \mathsf{ct})$
  $i \leftarrow i + 1$
  return ct, i
Decryption oracle $D^b_{\mathsf{sk}}(j) :=$
  if $S[j].m_0 = S[j].m_1$
    return $\mathtt{Dec}_{\mathsf{sk}}(S[j].\mathsf{ct})$
  else
    return $\perp$

Definition 5 (IND-CPA^{D} Security [44]). *Let $\Pi = (\mathtt{KeyGen}, \mathtt{Enc}, \mathtt{Dec}, \mathtt{Eval})$ be an HE scheme. Let θ denotes an admissible oracle query. The IND-CPA^{D} game is defined as an indistinguishability game parameterized by distribution ensembles $\{E^b_\theta, H^b_\theta, D^b_\theta\}_\theta$ for $b = 0$ or 1 of Alg. 1. We say that the HE scheme Π is IND-CPA^{D} secure, if any PPT adversary $\mathcal{A}$ has a negligible advantage $\mathsf{Adv}^{\mathsf{IND\text{-}CPA^D}}(\mathcal{A})$ against IND-CPA^{D} security defined as:*

$$|2 \cdot \Pr[b = b' | b \xleftarrow{\$} \{0,1\}, b' \leftarrow \mathcal{A}(\mathsf{pk}, \mathtt{Enc}(b))] - 1|.$$

B The t-out-of-n Threshold CKKS

We present a t-out-of-n threshold CKKS scheme. Our construction is instantiated using Shamir secret sharing, but the approach is flexible and can accommodate alternative secret sharing schemes when additional properties are required.

$\mathtt{ThCKKS.Setup}(1^\lambda)$ is identical to $\mathtt{ThCKKS.Setup}(1^\lambda)$ in Section 2.2. We present an ideal functionality of t-out-of-n threshold key generation in Fig.12 and briefly discuss how the threshold keys can be generated.

The t-out-of-n key generation functionality can be viewed as simply thresholdizing the secret key sk and the evaluation key $\mathbf{r}_1$ using Shamir secret sharing. We rely on a trusted third party (TTP) to generate the Shamir polynomial and distribute the Shamir shares. Since key generation is a one-time setup phase many existing threshold cryptosystems adopt this model.

Functionality $\mathcal{F}_{\text{ThKeyGen}}$

Setup:
Let pp be a public parameter for CKKS, a threshold t, and a total number of parties n.

1. Generate $(\mathsf{sk} = s, \mathsf{pk} = (b, a), \mathsf{evk}) \leftarrow \mathtt{CKKS.KeyGen}(1^\lambda)$.
2. Sample $\mathrm{P_s} \leftarrow \mathcal{R}_{PQ}[X]$, a degree $t-1$ polynomial such that $\mathrm{P_s}(0) = s$.
3. $(\mathsf{sk}_i = s_i)_{i\in[n]} := (\mathrm{P_s}(i))_{i\in[n]}$.
4. Output $(\mathsf{sk}_i, \mathsf{pk}, \mathsf{evk})$ to P_i for $1 \le i \le n$.

Fig. 12. Functionality of distributed (t, n) threshold key generation

The distributed decryption of a ciphertext can be carried out by incorporating Lagrange coefficients for Shamir reconstruction. For a reconstruction set $\mathcal{T} \subseteq [n]$ and each $i \in \mathcal{T}$, define the Lagrange coefficient for interpolation at zero as $\lambda_{\mathcal{T},i} := \prod_{j\in\mathcal{T}\setminus\{i\}} \frac{-j}{i-j}$.

• $\mathtt{ThCKKS.DistDec}(\{\mathsf{sk}_i\}_{i\in[n]}, \mathsf{ct})$: Given a ciphertext $\mathsf{ct} = (c_0, c_1) \in R^2_{Q_\ell}$, the distributed decryption protocol proceeds as follows:

1. Each party P_i samples $e_i \leftarrow \mathcal{D}$ where $\mathcal{D}$ denotes the smudging distribution, then publishes $\mu_i := c_1 \cdot \lambda_{\mathcal{T},i} \cdot s_i + e_i \pmod{Q_\ell}$.
2. All parties compute and output $m := c_0 + \sum_{i\in\mathcal{T}} \mu_i \pmod{Q_\ell}$.

C Simple Approximate 2PC Protocol

In this section, we provide a functionality of 2PC protocol where only the client has the input, a protocol realizing the functionality, and the security proof that the protocol satisfies standard MPC security.

We assume that the client has private input x, while the server acts solely as an evaluator of a public circuit C. After computing the ciphertext corresponding to $C(x)$, the server returns the result to the client, who decrypts it to obtain an approximate value of $C(x)$. The corresponding ideal functionality is illustrated in Fig. 13. At the end of the protocol, the client (P_1) discloses the result to the server (P_2), and both parties obtain the same output.

Functionality $\mathcal{F}_{\text{Simp2PC}}$

Setup:
Let C be an evaluation circuit and $\mathcal{D}$ be an error distribution. The functionality receives x from P_1 but nothing from P_2.

1. Compute $\hat{z} := C(x)$.
2. Sample $e \leftarrow \mathcal{D}$ and let $z := \hat{z} + e$.
3. Output z to both P_1 and P_2.

Fig. 13. Functionality of simple 2PC of approximations.

In the following, we show that the functionality $\mathcal{F}_{\text{Simp2PC}}$ can be realized using an approximate homomorphic encryption scheme such as CKKS. Specifically, the standard HE-based 2PC protocol, in which the client and the server act as the key owner and the evaluator, respectively, securely realizes the desired functionality by employing the noise smudging technique. Figure 14 depicts the protocol Π_{Simp2PC} that realizes this functionality. Throughout this section, we omit the prefix $\mathtt{CKKS}$ from the CKKS algorithms when it is clear from the context. We then prove that the protocol achieves the simulation-based security.

Protocol Π_{Simp2PC}

1. P_1 generates $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$. Then P_1 sends $(\mathsf{pk}, \mathsf{ct}_{\mathsf{in}})$ to P_2.
2. P_2 computes $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}})$ and sends $\mathsf{ct}_{\mathsf{out}}$ to P_1.
3. P_1 decrypts $z_{\mathsf{dec}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}})$, samples $e \leftarrow \mathcal{D}$, and computes $z_{\mathsf{out}} := z_{\mathsf{dec}} + e$.
4. P_1 sends z_{out} to P_2, and both parties output z_{out}.

Fig. 14. Simple 2PC protocols of approximations.

Theorem 6. *For a plaintext x and a circuit C, suppose that $\|x\| \leq B$ for some $B > 0$ and let $E := \mathsf{Est}(C, B)$. If* CKKS *is* IND-CPA *secure and $\mathcal{D}$ is an E-smudging distribution, then the 2PC protocol $\Pi_{Simp2PC}$ securely realizes the functionality $\mathcal{F}_{Simp2PC}$.*

Proof. Let us first show the correctness of Π_{Simp2PC}. Let $\hat{z} := C(x)$ and let $z_{\mathsf{dec}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}})$ be the decryption of the resulting ciphertext. By the definition of the function Est, we have $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ for some $e_{\mathsf{out}} \in R$ such that $\|e_{\mathsf{out}}\| \leq E$. Thus, the distribution of the output $z_{\mathsf{out}} = z_{\mathsf{dec}} + e = \hat{z} + (e_{\mathsf{out}} + e)$ with $e \leftarrow \mathcal{D}$ is statistically indistinguishable from the distribution of the output $z = \hat{z} + e$ in the functionality $\mathcal{F}_{\mathrm{Simp2PC}}$, since $\mathcal{D}$ is an E-smudging distribution.

To prove the security of Π_{Simp2PC}, we separately consider a corrupted client P_1 and a corrupted server P_2, using a sequence of hybrid arguments for each corruption case. For convenience, we use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted party's view and the protocol output. Since both parties output the same value, we identify the output vector $(z_{\mathsf{out}}, z_{\mathsf{out}})$ with the common output z_{out}.

We first consider a corrupted client P_1. The simulator $\mathtt{Sim}^1_{\mathrm{Simp2PC}}(x, z)$ proceeds as follows:

1. Generate $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$.
2. Compute $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}})$ and $z_{\mathsf{dec}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}})$.
3. Set $e := z - z_{\mathsf{dec}}$.
4. Output $(x, \mathsf{sk}, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}_{\mathsf{out}}, e)$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world execution. Its joint distribution is as follows:

$$
\begin{aligned}
&\{(\mathsf{view}_1^{\Pi_{\mathrm{Simp2PC}}}(x), \mathsf{output}^{\Pi_{\mathrm{Simp2PC}}}(x))\} \\
\equiv&\{((x, \mathsf{sk}, \mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}, \mathsf{ct}_{\mathsf{out}}, e), z_{\mathsf{dec}} + e) \mid (\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \\
&\quad \mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x), \mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}), z_{\mathsf{dec}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}}) = \hat{z} + e_{\mathsf{out}}, e \leftarrow \mathcal{D}\}.
\end{aligned}
$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that it replaces the smudging error e in both the corrupted client's view and the protocol output with $e - e_{\mathsf{out}}$. More precisely, it samples $e \leftarrow \mathcal{D}$, includes $e - e_{\mathsf{out}}$ in the corrupted client's view, and sets the protocol output to $z_{\mathsf{dec}} + (e - e_{\mathsf{out}}) = \hat{z} + e_{\mathsf{out}} + e - e_{\mathsf{out}} = \hat{z} + e$.

By approximate correctness, $\|e_{\mathsf{out}}\| \leq E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, the distributions of e and $e - e_{\mathsf{out}}$ are statistically indistinguishable. Therefore, $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{Simp2PC}}(x)$ instead of explicitly computing $\hat{z} = C(x)$ and sampling $e \leftarrow \mathcal{D}$. It then sets the smudging error in the corrupted client's view to $z - z_{\mathsf{dec}}$ and sets the protocol output to z.

By the definition of $\mathcal{F}_{\mathrm{Simp2PC}}$, its output is distributed as $z = \hat{z} + e$ for $e \leftarrow \mathcal{D}$. Consequently, $z - z_{\mathsf{dec}} = \hat{z} + e - (\hat{z} + e_{\mathsf{out}}) = e - e_{\mathsf{out}}$, which is precisely the smudging error included in the corrupted client's view in $\mathbf{Hyb}_1$. Therefore, $\mathbf{Hyb}_1 \equiv \mathbf{Hyb}_2$.

Note that $\mathbf{Hyb}_2$ is precisely the joint distribution produced by $\mathtt{Sim}^1_{\mathrm{Simp2PC}}(x, z)$ and the ideal functionality output z.

Next, we consider a corrupted server P_2. The simulator $\mathtt{Sim}^2_{\mathrm{Simp2PC}}(z)$ proceeds as follows:

1. Generate $(\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$ and $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
2. Output $(\mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}})$.

We construct the following hybrids.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world execution. Its joint distribution is as follows:

$$\begin{aligned}
&\{(\mathsf{view}_2^{\Pi_{\mathrm{Simp2PC}}}(x), \mathsf{output}^{\Pi_{\mathrm{Simp2PC}}}(x))\} \\
\equiv&\{((\mathsf{pk}, \mathsf{evk}, \mathsf{ct}_{\mathsf{in}}), \hat{z} + e_{\mathsf{out}} + e) \mid (\mathsf{sk}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \\
&\quad \mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x), \mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_{\mathsf{in}}), \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}}) = \hat{z} + e_{\mathsf{out}}, e \leftarrow \mathcal{D}\}.
\end{aligned}$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that it sets the protocol output to $z_1 := \hat{z} + e$ rather than $\hat{z} + e_{\mathsf{out}} + e$. By approximate correctness, $\|e_{\mathsf{out}}\| \leq E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, the distributions of $e_{\mathsf{out}} + e$ and e are statistically indistinguishable. The corrupted server's view is unchanged between the two hybrids. Therefore, $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is identical to $\mathbf{Hyb}_1$, except that it samples $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ instead of $\mathsf{ct}_{\mathsf{in}} \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x)$. By the IND-CPA security of CKKS, $\mathbf{Hyb}_1 \approx_c \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{Simp2PC}}(x)$ instead of explicitly computing $\hat{z} = C(x)$ and sampling $e \leftarrow \mathcal{D}$. By the definition of $\mathcal{F}_{\mathrm{Simp2PC}}$, its output is distributed as $z = \hat{z} + e$ for $e \leftarrow \mathcal{D}$. Therefore, $\mathbf{Hyb}_2 \equiv \mathbf{Hyb}_3$.

Note that $\mathbf{Hyb}_3$ is precisely the joint distribution produced by $\mathtt{Sim}^2_{\mathrm{Simp2PC}}(z)$ and the ideal functionality output z.

Combining the above hybrid arguments, Π_{Simp2PC} securely realizes $\mathcal{F}_{\mathrm{Simp2PC}}$. □

D Construction of t-out-of-n Threshold MPC of Approximations

We construct an approximate MPC protocol using threshold CKKS. We provide the ideal functionalities, protocols, and security proofs for protocols satisfying standard MPC security and liberal security. Our construction uses the t-out-of-n threshold CKKS scheme ThCKKS described in Appendix B. Throughout this section, we omit the prefix ThCKKS from the CKKS algorithms when it is clear from the context. To construct the approximate MPC protocol achieving simulation-based security, we first explain the functionality of t-out-of-n collaborative sampling in Fig. 15.

Functionality $\mathcal{F}_{\mathrm{ThColSamp}}$

Setup:
Let $\mathcal{D}$ be a smudging error distribution over R, and $\mathcal{T}$ the set of t decrypting parties.

1. Sample $e \leftarrow \mathcal{D}$.
2. Sample $r_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ for $1 \leq i \leq t-1$, and let $r_t := e - \sum_{i=1}^{t-1} r_i \pmod{q_{\mathsf{out}}}$.
3. Output r_i to party P_i for $i \in \mathcal{T}$.

Fig. 15. Functionality of the (t, n) collaborative sampling.

It can be realized by using the t-out-of-t bit shares in Sec. 4.3, once the set of decrypting parties has been determined. Similar to Sec. 4.3, we extend the bit-sharing protocol to sample from the desired smudging distribution $\mathcal{D}$.

The ideal functionality of the t-out-of-n approximate MPC protocol is given in Fig. 16 and the concrete protocol realizing $\mathcal{F}_{\mathrm{ThMPC}}$ is given in Fig. 17. Now, we show that the approximate MPC protocol Π_{ThMPC} is secure in the $\mathcal{F}_{\mathrm{ThColSamp}}$-hybrid model.

Functionality $\mathcal{F}_{\text{ThMPC}}$

Setup:
Let C be an evaluation circuit, $\mathcal{D}$ be an error distribution, and $\mathcal{T}$ the set of t decrypting parties. The functionality receives x_i from n parties P_i $(1 \le i \le n)$.

1. Compute $\hat{z} \leftarrow C(x_1, \ldots, x_n)$.
2. Sample $e \leftarrow \mathcal{D}$ and let $z := \hat{z} + e$.
3. Output z to all parties in $\mathcal{T}$.

Fig. 16. Functionality of the (t, n)-approximate MPC protocol.

Protocol Π_{ThMPC}

1. Agree on a set of participating parties $\mathcal{T}$.
2. All parties generate $(\mathsf{sk}_i, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
3. Each party P_i encrypts $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$ and broadcasts it.
4. All parties compute $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, (\mathsf{ct}_i)_{1 \le i \le n})$.
   We denote $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}})$.
5. The parties call $\mathcal{F}_{\text{ThColSamp}}$ and each party P_i receives $r_i \in R_{q_{\mathsf{out}}}$ for $i \in \mathcal{T}$.
6. Each party P_i computes $u_i = a_{\mathsf{out}} \cdot \lambda_{\mathcal{T},i} \cdot s_i + r_i \pmod{q_{\mathsf{out}}}$ and broadcasts it.
7. All parties compute and output $z = b_{\mathsf{out}} + \sum_{i \in \mathcal{T}} u_i \pmod{q_{\mathsf{out}}}$.

Fig. 17. A secure (t, n)-approximate MPC protocol for evaluating a circuit C.

Theorem 7. *For plaintexts x_i and a circuit C, suppose that $\|x_i\| \le B_i$ for some $B_i > 0$ and let $E := \mathsf{Est}(C, \{B_i\}_{i \le n})$. If $\mathtt{ThCKKS}$ is $\mathsf{IND\text{-}CPA}$ secure and $\mathcal{D}$ is an E-smudging distribution, then the MPC protocol Π_{ThMPC} securely realizes the functionality $\mathcal{F}_{ThMPC}$ in the $\mathcal{F}_{ThColSamp}$-hybrid model in the standard sense.*

Proof. We first guarantee the correctness of Π_{ThMPC}. Let $\hat{z} := C(x_1, \ldots, x_n)$. From the error estimator Est, we have $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ for some $e_{\mathsf{out}} \in R$ such that $\|e_{\mathsf{out}}\| \le E$. By the linearity of the decryption process, the output can be represented as follows:

$$\begin{aligned} z_{\mathsf{out}} &= b_{\mathsf{out}} + \sum_{i \in \mathcal{T}} u_i = b_{\mathsf{out}} + \sum_{i \in \mathcal{T}} a_{\mathsf{out}} \cdot \lambda_{\mathcal{T},i} \cdot s_i \pmod{q_{\mathsf{out}}} \\ &= z_{\mathsf{dec}} + e = \hat{z} + e_{\mathsf{out}} + e \end{aligned}$$

for some $e \leftarrow \mathcal{D}$. Since $\mathcal{D}$ is an E-smudging distribution, we can conclude that the distribution of the protocol's output is statistically indistinguishable from the distribution of the output $z + e$ in the functionality $\mathcal{F}_{\text{NewMPC}}$.

To prove the security of Π_{ThMPC}, we consider the case where the adversary corrupts $t - 1$ parties. Let $A \subsetneq \mathcal{T}$ be the set of corrupted parties and let $H := [n] \setminus A$ be the set of honest parties. Let $\mathcal{T}$ be the reconstruction set used in the protocol, and fix an honest party $h \in H \cap \mathcal{T}$. We use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted parties' view and the protocol output.

The simulator $\mathtt{Sim}^A_{\text{ThMPC}}((x_i)_{i \in A}, z)$ proceeds as follows:

1. Generate the threshold keys $((s_i)_{i \in [n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. For each $i \in A$, compute $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$; for each $i \in H$, compute $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
3. Compute $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$.
4. Sample $r_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ and set $u_i := a_{\mathsf{out}} \lambda_{\mathcal{T},i} s_i + r_i$ for every $i \in A$.
5. Sample $u_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ for every $i \in (H \cap \mathcal{T}) \setminus \{h\}$.
6. Set $u_h := z - b_{\mathsf{out}} - \sum_{i \in \mathcal{T} \setminus \{h\}} u_i \pmod{q_{\mathsf{out}}}$.

7. Output $((x_i, r_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, (u_i)_{i\in H\cap\mathcal{T}})$.

We now construct the hybrids.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world execution. Its joint distribution is:

$$\begin{aligned}
&\{(\mathsf{view}_A^{\Pi_{\mathrm{ThMPC}}}(\boldsymbol{x}), \mathsf{output}^{\Pi_{\mathrm{ThMPC}}}(\boldsymbol{x}))\}\\
\equiv&\{((x_i, r_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, (u_i)_{i\in H\cap\mathcal{T}}), \hat{z} + e_{\mathsf{out}} + e) \mid\\
&\quad((s_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i) \text{ for all } i \in [n],\\
&\quad\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n), \quad (r_i)_{i\in\mathcal{T}} \leftarrow \mathcal{F}_{\mathrm{ThColSamp}},\\
&\quad u_i := a_{\mathsf{out}}\lambda_{\mathcal{T},i}s_i + r_i \text{ for all } i \in \mathcal{T}, \quad e := \textstyle\sum_{i\in\mathcal{T}} r_i\}.
\end{aligned}$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that it samples $u_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ independently for every $i \in (H \cap \mathcal{T}) \setminus \{h\}$ and defines $u_h := \hat{z} + e_{\mathsf{out}} + e - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i$.

For every honest party $i \in H \cap \mathcal{T}$, the value $u_i = a_{\mathsf{out}}\lambda_{\mathcal{T},i}s_i + r_i$ is uniform over $R_{q_{\mathsf{out}}}$ since r_i is uniformly sampled by $\mathcal{F}_{\mathrm{ThColSamp}}$. Moreover, the relation $\hat{z} + e_{\mathsf{out}} + e = b_{\mathsf{out}} + \sum_{i\in\mathcal{T}} u_i$ holds in the real protocol. Therefore, the distribution of the honest parties' partial decryptions is unchanged, and $\mathbf{Hyb}_0 \equiv \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is identical to $\mathbf{Hyb}_1$, except that it replaces the effective error $e_{\mathsf{out}} + e$ with e. More precisely, it samples $e \leftarrow \mathcal{D}$, defines the final honest partial decryption as $u_h := \hat{z} + e - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i$, and sets the protocol output to $\hat{z} + e$.

By approximate correctness, $\|e_{\mathsf{out}}\| \le E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, $\mathbf{Hyb}_1 \approx_s \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid is identical to $\mathbf{Hyb}_2$, except that, for every honest party $i \in H$, it samples $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ instead of $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$. The ciphertexts corresponding to corrupted parties remain encryptions of their actual inputs.

By the IND-CPA security of ThCKKS, the vector of ciphertexts encrypting the honest inputs is computationally indistinguishable from a vector of encryptions of zero. Hence, $\mathbf{Hyb}_2 \approx_c \mathbf{Hyb}_3$.

$\mathbf{Hyb}_4$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{ThMPC}}(\boldsymbol{x})$ instead of explicitly computing $\hat{z} = C(x_1, \ldots, x_n)$ and sampling $e \leftarrow \mathcal{D}$. It then sets $u_h := z - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i$. By the definition of $\mathcal{F}_{\mathrm{ThMPC}}$, its output is distributed as $z = \hat{z} + e$ for $e \leftarrow \mathcal{D}$. Therefore, $\mathbf{Hyb}_3 \equiv \mathbf{Hyb}_4$.

Finally, $\mathbf{Hyb}_4$ is precisely the joint distribution produced by $\mathtt{Sim}^A_{\mathrm{ThMPC}}((x_i)_{i\in A}, z)$ and the ideal functionality output z. Combining the above hybrid transitions, we conclude that Π_{ThMPC} securely realizes $\mathcal{F}_{\mathrm{ThMPC}}$ in the $\mathcal{F}_{\mathrm{ThColSamp}}$-hybrid model. □

We also prove that existing t-out-of-n approximate MPC protocol achieves the liberal security. The ideal functionality of this protocol is given in Fig. 18 and the concrete protocol realizing $\mathcal{F}_{\mathrm{PrevThMPC}}$ is given in Fig. 19.

Functionality $\mathcal{F}_{\mathrm{PrevThMPC}}$

Setup:
Let C be an evaluation circuit and $\mathcal{D}$ be an error distribution. The functionality receives x_i from n parties P_i $(1 \le i \le n)$.

1. Compute $\hat{z} = C((x_i)_{\mathcal{T}})$.
2. For $i \in \mathcal{T}$, sample $e_i \leftarrow \mathcal{D}$ and let $z := \hat{z} + \sum_{i\in\mathcal{T}} e_i$.
3. Output z to all parties.

Fig. 18. Functionality of the existing (t, n)-approximate MPC protocol.

Protocol $\Pi_{\mathrm{PrevThMPC}}$

1. Agree on a set of participating parties $\mathcal{T}$.
2. All parties generate $(\mathsf{sk}_i, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
3. Each party P_i encrypts $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$ and broadcasts it.
4. All parties compute $\mathsf{ct}_{\mathsf{out}} \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, (\mathsf{ct}_i)_{i\in\mathcal{T}})$.
   We denote $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}})$.
5. Each party P_i samples $e_i \leftarrow \mathcal{D}$, computes $u_i = a_{\mathsf{out}} \cdot \lambda_{\mathcal{T},i} \cdot s_i + e_i \pmod{q_{\mathsf{out}}}$ and broadcasts it.
6. All parties compute and output $z_{\mathsf{out}} = b_{\mathsf{out}} + \sum_{i\in\mathcal{T}} u_i \pmod{q_{\mathsf{out}}}$.

Fig. 19. Existing (t, n)-approximate MPC protocol for evaluating a circuit C.

Theorem 8. *For plaintexts x's and a circuit C, suppose that $\|x_i\| \le B_i$ for some $B_i > 0$ and let $E := \mathsf{Est}(C, \{B_i\}_{i\le n})$. If $\mathtt{ThCKKS}$ is $\mathsf{IND\text{-}CPA}$ secure and $\mathcal{D}$ is an E-smudging distribution, then the MPC protocol $\Pi_{PrevThMPC}$ securely realizes the functionality $\mathcal{F}_{PrevThMPC}$ in the liberal sense.*

Proof. We first show the correctness of $\Pi_{\mathrm{PrevThMPC}}$. Let $\hat{z} := C((x_i)_{i\in\mathcal{T}})$ and $z_{\mathsf{dec}} := \mathtt{Dec}_{\mathsf{sk}}(\mathsf{ct}_{\mathsf{out}}) := b_{\mathsf{out}} + \sum_{i\in\mathcal{T}} a_{\mathsf{out}} \cdot \lambda_{\mathcal{T},i} \cdot s_i$. From the error estimator Est, we have $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ for some $e_{\mathsf{out}} \in R$ such that $\|e_{\mathsf{out}}\| \le E$. By the linearity of the decryption process, the output can be represented as follows:

$$\begin{aligned} z_{\mathsf{out}} &= b_{\mathsf{out}} + \sum_{i\in\mathcal{T}} u_i = b_{\mathsf{out}} + \sum_{i\in\mathcal{T}} (a_{\mathsf{out}} \cdot \lambda_{\mathcal{T},i} \cdot s_i + e_i) \pmod{q_{\mathsf{out}}} \\ &= z_{\mathsf{dec}} + e = \hat{z} + e_{\mathsf{out}} + e, \end{aligned}$$

for some $e \leftarrow \mathcal{D}$. Since $\mathcal{D}$ is an E-smudging distribution, we can conclude that the distribution of the protocol's output is statistically indistinguishable from the distribution of the output $z = \hat{z} + e$ in the functionality $\mathcal{F}_{\mathrm{ThMPC}}$.

To prove the liberal security of $\Pi_{\mathrm{PrevThMPC}}$, we consider the case where the adversary corrupts $t-1$ parties. Let P_h be the unique honest party, let $A := \mathcal{T} \setminus \{h\}$, and assume that $h \in \mathcal{T}$. For convenience, we use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted parties' view and the protocol output.

The simulator $\mathtt{Sim}^A_{\mathrm{PrevThMPC}}((x_i)_{i\in A}, \hat{z}, z)$ proceeds as follows:

1. Generate threshold keys $((s_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. For each $i \in A$, compute $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$, and compute $\mathsf{ct}_h \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
3. Compute $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$.
4. Let $e := z - \hat{z}$ and sample $(e_i)_{i\in\mathcal{T}} \leftarrow \{(e_i)_{i\in\mathcal{T}} \leftarrow \mathcal{D}^{|\mathcal{T}|} \mid \sum_{i\in\mathcal{T}} e_i = e\}$.
5. For every $i \in A$, set $u_i := a_{\mathsf{out}} \lambda_{\mathcal{T},i} s_i + e_i$.
6. Set $u_h := z - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i \pmod{q_{\mathsf{out}}}$.
7. Output $((x_i, e_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, u_h)$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world execution. Its joint distribution is

$$\begin{aligned} &\{(\mathsf{view}_A^{\Pi_{\mathrm{PrevThMPC}}}(\boldsymbol{x}), \mathsf{output}^{\Pi_{\mathrm{PrevThMPC}}}(\boldsymbol{x}))\} \\ \equiv &\{((x_i, e_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, u_h), \hat{z} + e_{\mathsf{out}} + e) \mid \\ &\quad ((s_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i) \text{ for all } i \in [n], \\ &\quad \mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n), \quad e_i \leftarrow \mathcal{D} \text{ for all } i \in \mathcal{T}, \\ &\quad u_i := a_{\mathsf{out}} \lambda_{\mathcal{T},i} s_i + e_i \text{ for all } i \in \mathcal{T}, \quad e := \textstyle\sum_{i\in\mathcal{T}} e_i\}. \end{aligned}$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that the honest party's partial decryption and the protocol output are shifted by $-e_{\mathsf{out}}$. More precisely, it sets $u_h := a_{\mathsf{out}} \lambda_{\mathcal{T},h} s_h + e_h - e_{\mathsf{out}}$ and outputs $\hat{z} + e$.

By approximate correctness, $\|e_{\mathsf{out}}\| \le E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, the distributions of e_h and $e_h - e_{\mathsf{out}}$ are statistically indistinguishable. Consequently, $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is an algebraic rewriting of $\mathbf{Hyb}_1$. Since $z_{\mathsf{dec}} = \hat{z} + e_{\mathsf{out}}$ and $z_{\mathsf{dec}} = b_{\mathsf{out}} + \sum_{i\in\mathcal{T}} a_{\mathsf{out}}\lambda_{\mathcal{T},i}s_i$, we have

$$\begin{aligned} u_h &= a_{\mathsf{out}}\lambda_{\mathcal{T},h}s_h + e_h - e_{\mathsf{out}} \\ &= \hat{z} + e - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i. \end{aligned}$$

Thus, this hybrid samples the corrupted parties' values as before and sets $u_h := \hat{z} + e - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i$. No distributional change occurs, and $\mathbf{Hyb}_1 \equiv \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid is identical to $\mathbf{Hyb}_2$, except that it samples $\mathsf{ct}_h \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ instead of $\mathsf{ct}_h \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_h)$. The ciphertexts ct_i for $i \in A$ remain encryptions of the corrupted parties' actual inputs.

By the IND-CPA security of ThCKKS, the resulting ciphertext vector, and hence the corrupted parties' view, is computationally indistinguishable from that in $\mathbf{Hyb}_2$. Therefore, $\mathbf{Hyb}_2 \approx_c \mathbf{Hyb}_3$.

$\mathbf{Hyb}_4$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{PrevThMPC}}(\boldsymbol{x})$ instead of explicitly computing $\hat{z} = C((x_i)_{i\in[n]})$ and sampling $(e_i)_{i\in\mathcal{T}} \leftarrow \mathcal{D}^{|\mathcal{T}|}$. It computes $e := z - \hat{z}$ and samples $(e_i)_{i\in\mathcal{T}}$ conditioned on $\sum_{i\in\mathcal{T}} e_i = e$. It then sets $u_h := z - b_{\mathsf{out}} - \sum_{i\in\mathcal{T}\setminus\{h\}} u_i$.

By the definition of $\mathcal{F}_{\mathrm{PrevThMPC}}$, its output is distributed as $z = \hat{z} + \sum_{i\in\mathcal{T}} e_i$ for independently sampled $e_i \leftarrow \mathcal{D}$. Thus, sampling $(e_i)_{i\in\mathcal{T}}$ conditioned on $\sum_{i\in\mathcal{T}} e_i = z - \hat{z}$ is an equivalent reparameterization of the distribution in $\mathbf{Hyb}_3$, and $\mathbf{Hyb}_3 \equiv \mathbf{Hyb}_4$.

Finally, $\mathbf{Hyb}_4$ is precisely the joint distribution produced by $\mathtt{Sim}^A_{\mathrm{PrevThMPC}}((x_i)_{i\in A}, \hat{z}, z)$ and the ideal functionality output z. Combining the above hybrid transitions, $\Pi_{\mathrm{PrevThMPC}}$ securely realizes $\mathcal{F}_{\mathrm{PrevThMPC}}$ in the liberal sense. □

E Proof of Theorem 3

In this section, we provide a security proof of Theorem 3.

Proof. To prove the security of Π_{NewMPC}, let $A \subsetneq [n]$ and $H := [n] \setminus A$ be the sets of corrupted and honest parties, respectively. Fix an honest party $h \in H$. For convenience, we use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted parties' view and the protocol output.

The simulator $\mathtt{Sim}^A_{\mathrm{NewMPC}}((x_i)_{i\in A}, z)$ proceeds as follows:

1. Generate $((s_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. For every $i \in A$, compute $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$; for every $i \in H$, compute $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
3. Compute $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$.
4. Sample $r_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ and set $u_i := a_{\mathsf{out}}s_i + r_i$ for every $i \in A$.
5. Sample $u_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ for every $i \in H \setminus \{h\}$.
6. Set $u_h := z - b_{\mathsf{out}} - \sum_{i\in[n]\setminus\{h\}} u_i \pmod{q_{\mathsf{out}}}$.
7. Output $((x_i, r_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, (u_i)_{i\in H})$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world execution. Its joint distribution is

$$\begin{aligned} &\{(\mathsf{view}_A^{\Pi_{\mathrm{NewMPC}}}(\boldsymbol{x}), \mathsf{output}^{\Pi_{\mathrm{NewMPC}}}(\boldsymbol{x}))\} \\ \equiv &\{((x_i, r_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, (u_i)_{i\in H}), \hat{z} + e_{\mathsf{out}} + e) \mid \\ &\quad ((s_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i) \text{ for all } i \in [n], \\ &\quad \mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n), \quad (r_i)_{i\in[n]} \leftarrow \mathcal{F}_{n\text{-ColSamp}}, \\ &\quad u_i := a_{\mathsf{out}}s_i + r_i \text{ for all } i \in [n], \quad e := \textstyle\sum_{i\in[n]} r_i\}. \end{aligned}$$

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that it samples $u_i \xleftarrow{\$} R_{q_{\mathsf{out}}}$ independently for every $i \in H \setminus \{h\}$ and defines $u_h := \hat{z} + e_{\mathsf{out}} + e - b_{\mathsf{out}} - \sum_{i\in[n]\setminus\{h\}} u_i$.

For each honest party $i \in H$, the value $u_i = a_{\mathsf{out}} s_i + r_i$ is uniform over $R_{q_{\mathsf{out}}}$ because r_i is uniformly sampled by $\mathcal{F}_{n\text{-ColSamp}}$. Moreover, in the real protocol, $\hat{z}+e_{\mathsf{out}}+e = b_{\mathsf{out}}+\sum_{i\in[n]} u_i$. Thus, the distribution of the honest parties' partial decryptions is unchanged, and $\mathbf{Hyb}_0 \equiv \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is identical to $\mathbf{Hyb}_1$, except that it replaces the effective error $e_{\mathsf{out}} + e$ with e. More precisely, it sets the output to $\hat{z} + e$ and defines $u_h := \hat{z} + e - b_{\mathsf{out}} - \sum_{i\in[n]\setminus\{h\}} u_i$.

By approximate correctness, $\|e_{\mathsf{out}}\| \le E$ with overwhelming probability. Since $\mathcal{D}$ is an E-smudging distribution, $\mathbf{Hyb}_1 \approx_s \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid is identical to $\mathbf{Hyb}_2$, except that it samples $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ for every honest party $i \in H$ instead of $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$. The ciphertexts corresponding to corrupted parties remain encryptions of their actual inputs.

By the IND-CPA security of CKKS, the vector of ciphertexts encrypting the honest inputs is computationally indistinguishable from a vector of encryptions of zero. Hence, $\mathbf{Hyb}_2 \approx_c \mathbf{Hyb}_3$.

$\mathbf{Hyb}_4$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{NewMPC}}(\boldsymbol{x})$ instead of explicitly computing $\hat{z} = C(x_1, \ldots, x_n)$ and sampling $e \leftarrow \mathcal{D}$. It then defines $u_h := z - b_{\mathsf{out}} - \sum_{i\in[n]\setminus\{h\}} u_i$.

By the definition of $\mathcal{F}_{\mathrm{NewMPC}}$, its output is distributed as $z = \hat{z} + e$ for $e \leftarrow \mathcal{D}$. Therefore, $\mathbf{Hyb}_3 \equiv \mathbf{Hyb}_4$.

Finally, $\mathbf{Hyb}_4$ is precisely the joint distribution produced by $\mathtt{Sim}^A_{\mathrm{NewMPC}}((x_i)_{i\in A}, z)$ and the ideal functionality output z. Combining the above hybrid transitions, Π_{NewMPC} securely realizes $\mathcal{F}_{\mathrm{NewMPC}}$ in the $\mathcal{F}_{n\text{-ColSamp}}$-hybrid model. □

F Proof of Theorem 5

In this section, we provide a security proof of Theorem 5.

Proof. To prove the liberal security of Π_{PrevMPC}, we consider the case where the adversary corrupts exactly $n-1$ parties. Let P_h be the unique honest party and let $A := [n] \setminus \{h\}$. For convenience, we use $\mathbf{Hyb}_i$ to denote both the i-th hybrid experiment and the joint distribution it induces over the corrupted parties' view and the protocol output.

The simulator $\mathtt{Sim}^A_{\mathrm{PrevMPC}}((x_i)_{i\in A}, \hat{z}, z)$ proceeds as follows:

1. Generate $((s_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)$.
2. For every $i \in A$, compute $\mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i)$; for the honest party P_h, compute $\mathsf{ct}_h \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$.
3. Compute $\mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n)$.
4. Let $e := z - \hat{z}$ and sample $(e_i)_{i\in[n]} \leftarrow \{(e_i)_{i\in[n]} \leftarrow \mathcal{D}^n \mid \sum_{i\in[n]} e_i = e\}$.
5. Set $u_i := a_{\mathsf{out}} s_i + e_i$ for every $i \in A$.
6. Set $u_h := z - b_{\mathsf{out}} - \sum_{i\in A} u_i \pmod{q_{\mathsf{out}}}$.
7. Output $((x_i, e_i, s_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, u_h)$.

The main idea is to construct the honest party's partial decryption u_h from the ideal output z and the corrupted parties' partial decryptions, thereby ensuring that $b_{\mathsf{out}} + \sum_{i\in[n]} u_i = z$.

We proceed as follows.

$\mathbf{Hyb}_0$: This hybrid corresponds to the real-world execution. Its joint distribution is

$$
\begin{aligned}
&\{(\mathsf{view}_A^{\Pi_{\mathrm{PrevMPC}}}(\boldsymbol{x}), \mathsf{output}^{\Pi_{\mathrm{PrevMPC}}}(\boldsymbol{x}))\} \\
\equiv&\{((x_i, e_i, \mathsf{sk}_i)_{i\in A}, \mathsf{pk}, \mathsf{evk}, (\mathsf{ct}_i)_{i\in[n]}, \mathsf{ct}_{\mathsf{out}}, u_h), \hat{z} + e_{\mathsf{out}} + e) \mid \\
&\quad ((\mathsf{sk}_i)_{i\in[n]}, \mathsf{pk}, \mathsf{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \mathsf{ct}_i \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_i) \text{ for all } i \in [n], \\
&\quad \mathsf{ct}_{\mathsf{out}} := (b_{\mathsf{out}}, a_{\mathsf{out}}) \leftarrow \mathtt{Eval}_{\mathsf{evk}}(C, \mathsf{ct}_1, \ldots, \mathsf{ct}_n), \\
&\quad e_i \leftarrow \mathcal{D} \text{ for all } i \in [n], \quad u_i := a_{\mathsf{out}} s_i + e_i \text{ for all } i \in [n], \\
&\quad e := \textstyle\sum_{i\in[n]} e_i\}.
\end{aligned}
$$

Here, the output satisfies $\hat{z} + e_{\mathsf{out}} + e = b_{\mathsf{out}} + \sum_{i\in[n]} u_i$.

$\mathbf{Hyb}_1$: This hybrid is identical to $\mathbf{Hyb}_0$, except that the honest party's smudging error e_h is replaced by $e_h - e_{\mathsf{out}}$. Accordingly, it defines $u_h := a_{\mathsf{out}} s_h + e_h - e_{\mathsf{out}}$ and sets the protocol output to $\hat{z} + e$.

By approximate correctness, $\|e_{\mathsf{out}}\| \le E$ with overwhelming probability. Conditioned on all values sampled before e_h, the only difference between the two hybrids is whether the honest party's error is distributed as $\mathcal{D}$ or as $\mathcal{D} - e_{\mathsf{out}}$. Since $\mathcal{D}$ is an E-smudging distribution, $\mathbf{Hyb}_0 \approx_s \mathbf{Hyb}_1$.

$\mathbf{Hyb}_2$: This hybrid is an algebraic rewriting of $\mathbf{Hyb}_1$. Since $z_{\mathsf{dec}} = b_{\mathsf{out}} + \sum_{i\in[n]} a_{\mathsf{out}} s_i = \hat{z} + e_{\mathsf{out}}$, we have

$$\begin{aligned} u_h &= a_{\mathsf{out}} s_h + e_h - e_{\mathsf{out}} \\ &= \hat{z} + e - b_{\mathsf{out}} - \sum_{i\in A} u_i. \end{aligned}$$

Therefore, the honest party's partial decryption can equivalently be generated as $u_h := \hat{z} + e - b_{\mathsf{out}} - \sum_{i\in A} u_i$. No distributional change occurs, and hence $\mathbf{Hyb}_1 \equiv \mathbf{Hyb}_2$.

$\mathbf{Hyb}_3$: This hybrid is identical to $\mathbf{Hyb}_2$, except that it samples $\mathsf{ct}_h \leftarrow \mathtt{Enc}_{\mathsf{pk}}(0)$ instead of $\mathsf{ct}_h \leftarrow \mathtt{Enc}_{\mathsf{pk}}(x_h)$. The ciphertexts ct_i for $i \in A$ remain encryptions of the corrupted parties' actual inputs.

By the IND-CPA security of CKKS, the encryption of x_h is computationally indistinguishable from an encryption of zero. Since $\mathsf{ct}_{\mathsf{out}}$ and all subsequent values are efficiently computed from these ciphertexts, it follows that $\mathbf{Hyb}_2 \approx_c \mathbf{Hyb}_3$.

$\mathbf{Hyb}_4$: This hybrid obtains $z \leftarrow \mathcal{F}_{\mathrm{PrevMPC}}(\boldsymbol{x})$ and the liberal leakage $\hat{z} = C(x_1, \ldots, x_n)$. It computes $e := z - \hat{z}$ and samples $(e_i)_{i\in[n]}$ according to $\mathcal{D}^n$ conditioned on $\sum_{i\in[n]} e_i = e$. It then computes $u_i := a_{\mathsf{out}} s_i + e_i$ for every $i \in A$ and sets $u_h := z - b_{\mathsf{out}} - \sum_{i\in A} u_i$.

By the definition of $\mathcal{F}_{\mathrm{PrevMPC}}$, its output is generated as $z = \hat{z} + \sum_{i\in[n]} e_i$ for independently sampled $e_i \leftarrow \mathcal{D}$. Thus, first sampling z and then sampling the errors conditioned on $\sum_{i\in[n]} e_i = z - \hat{z}$ is an equivalent reparameterization of the distribution in $\mathbf{Hyb}_3$. Therefore, $\mathbf{Hyb}_3 \equiv \mathbf{Hyb}_4$.

Finally, $\mathbf{Hyb}_4$ is precisely the joint distribution produced by $\mathtt{Sim}^{A}_{\mathrm{PrevMPC}}((x_i)_{i\in A}, \hat{z}, z)$ and the ideal functionality output z. Combining the above hybrid transitions, Π_{PrevMPC} securely realizes $\mathcal{F}_{\mathrm{PrevMPC}}$ in the liberal sense. □